



index : src

FreeBSD source tree

main [about](#) [summary](#) [refs](#) [log](#) [tree](#) [commit](#) [diff](#)log msg path: [root/sys/kern/syscalls.master](#)blob: 709b01f0abbe8cfecdf9db282025c23b0c013b52 ([plain](#)) ([blame](#))

```

1 ; System call name/number master file.
2 ; Processed to created init_sysent.c, syscalls.c and syscall.h.
3
4 ; New FreeBSD system calls should be added to the bottom of this file.
5
6 ; Columns: number audit type name alt{name,tag,rtyp}/comments
7 ;   number   system call number, must be in order
8 ;   audit    the audit event associated with the system call
9 ;           A value of AUE_NULL means no auditing, but it also means that
10 ;           there is no audit event for the call at this time. For the
11 ;           case where the event exists, but we don't want auditing, the
12 ;           event should be #defined to AUE_NULL in audit_kevents.h.
13 ;   type     one of STD, OBSOL, RESERVED, UNIMPL, SYSMUX, COMPAT*,
14 ;           NODEF, NOARGS, NOPROTO, NOSTD
15 ;           The COMPAT* options may be combined with one or more NO*
16 ;           options separated by '|' with no spaces (e.g. COMPAT|NOARGS)
17 ;           The CAPENABLED option may be ORed into a type.
18 ;   name     pseudo-prototype of syscall routine
19 ;           If one of the following alts is different, then all appear:
20 ;   altname   name of system call if different
21 ;   alttag    name of args struct tag if different from [o]`name'`_args"
22 ;   altrtyp   return type if not int (bogus - syscalls always return int)
23 ;           for UNIMPL/OBSOL, name continues with comments
24
25 ; types:
26 ;   STD      always included
27 ;   COMPAT   included on COMPAT #ifdef
28 ;   COMPAT4  included on COMPAT_FREEBSD4 #ifdef (FreeBSD 4 compat)
29 ;   COMPAT6  included on COMPAT_FREEBSD6 #ifdef (FreeBSD 6 compat)
30 ;   COMPAT7  included on COMPAT_FREEBSD7 #ifdef (FreeBSD 7 compat)
31 ;   COMPAT10 included on COMPAT_FREEBSD10 #ifdef (FreeBSD 10 compat)
32 ;   COMPAT11 included on COMPAT_FREEBSD11 #ifdef (FreeBSD 11 compat)
33 ;   COMPAT12 included on COMPAT_FREEBSD12 #ifdef (FreeBSD 12 compat)
34 ;   COMPAT13 included on COMPAT_FREEBSD13 #ifdef (FreeBSD 13 compat)
35 ;   COMPAT14 included on COMPAT_FREEBSD14 #ifdef (FreeBSD 14 compat)
36 ;   OBSOL    obsolete, not included in system, only specifies name
37 ;   RESERVED reserved for local or vendor use (not for FreeBSD)
38 ;   UNIMPL   not implemented, placeholder only
39 ;   NOSTD    implemented but as a lkm that can be statically
40 ;           compiled in; sysent entry will be filled with lkmressys
41 ;           so the SYSCALL_MODULE macro works
42 ;   NOARGS   same as STD except do not create structure in sys/sysproto.h
43 ;   NODEF    same as STD except only have the entry in the syscall table
44 ;           added. Meaning - do not create structure or function
45 ;           prototype in sys/sysproto.h
46 ;   NOPROTO  same as STD except do not create structure or
47 ;           function prototype in sys/sysproto.h. Does add a
48 ;           definition to syscall.h besides adding a sysent.
49 ;   NOTSTATIC syscall is loadable
50 ;   SYSMUX   syscall multiplexer. No prototype, argument struct, or
51 ;           handler is declared or used. Handled in MD syscall code.
52 ;   CAPENABLED syscall is allowed in capability mode
53 ;
54 ; To support programmatic generation of both the default ABI and 32-bit compat
55 ; (freebsd32) we impose a number of restrictions on the types of system calls.
56 ; For integer types:
57 ; - Bare int and long are allowed (long is a sign of a bad interface).
58 ; - Use u_int and u_long rather than "unsigned (int|long)".
59 ; - size_t is allowed.
60 ; - typedefs are allowed, but new signed types that vary between 32- and
61 ;   64-bit ABIs must be added to makesyscalls.lua so it knows they require
62 ;   handling.
63 ; - Always-64-bit types other than dev_t, id_t, and off_t must be added to
64 ;   makesyscalls.lua.
65 ; For pointers:
66 ; - Prefer structs to typedefs so an ABI-specific suffix (e.g., "32") can

```

```

67 ; be prepended (e.g., ucontext_t -> struct ucontext -> struct ucontext32).
68 ; - Pointers to objects (structs, unions, etc) containing any long, pointer,
69 ; or time_t arguments need _Contains_ annotations. Such objects should be
70 ; padded such that all 64-bit types are 64-bit aligned.
71
72 ; annotations:
73 ; SAL 2.0 annotations are used to specify how system calls treat
74 ; arguments that are passed using pointers. There are three basic
75 ; annotations.
76 ;
77 ; _In_ Object pointed to will be read and not modified.
78 ; _Out_ Object pointed to will be written and not read.
79 ; _Inout_ Object pointed to will be written and read.
80 ;
81 ; These annotations are used alone when the pointer refers to a single
82 ; object i.e. scalar types, structs, and pointers, and not NULL. Adding
83 ; the _opt_ suffix, e.g. _In_opt_, implies that the pointer may also
84 ; refer to NULL.
85 ;
86 ; For pointers to arrays, additional suffixes are added:
87 ;
88 ; _In_z_, _Out_z_, _Inout_z_:
89 ; for a NUL terminated array e.g. a string.
90 ; _In_reads_z_(n), _Out_writes_z_(n), _Inout_updates_z_(n):
91 ; for a NUL terminated array e.g. a string, of known length n bytes.
92 ; _In_reads_(n), _Out_writes_(n), _Inout_updates_(n):
93 ; for an array of n elements.
94 ; _In_reads_bytes_(n), _Out_writes_bytes_(n), _Inout_updates_bytes_(n):
95 ; for a buffer of n-bytes.
96 ;
97 ; In addition to SAL annotations, pointers are annotated to indicate
98 ; that they point to types that change between ABIs. That means that
99 ; they contain long, pointer, or time_t types. This is indicated with
100 ; a _Contains_ annotation followed immediately by one or more of:
101 ;
102 ; long_ Object contains a direct (or typedef'd) long value and varies
103 ; between 32- and 64-bit ABIs. This includes size_t.
104 ; ptr_ Object contains pointers (or intptr_t) and varies between
105 ; 32- and 64-bit ABIs.
106 ; timet_ Object contains a time_t and varies between i386 and other
107 ; ABIs.
108
109 ; #include's, #defines's, etc. may be included, and are copied to the output
110 ; files. However, #ifdef, etc will be copied, but any lines that don't start
111 ; with # will not. Caveat Emptor.
112
113 #include <sys/param.h>
114 #include <sys/sysent.h>
115 #include <sys/sysproto.h>
116 %%ABI_HEADERS%%
117
118 0 AUE_NULL SYSMUX {
119     int syscall(
120         int number,
121         ...
122     );
123 }
124 1 AUE_EXIT STD|CAPENABLED {
125     void exit(
126         int rval
127     );
128 }
129 2 AUE_FORK STD|CAPENABLED {
130     int fork(void);
131 }
132 3 AUE_READ STD|CAPENABLED {
133     ssize_t read(
134         int fd,
135         _Out_writes_bytes_(nbyte) void *buf,
136         size_t nbyte
137     );
138 }
139 4 AUE_WRITE STD|CAPENABLED {
140     ssize_t write(
141         int fd,
142         _In_reads_bytes_(nbyte) const void *buf,
143         size_t nbyte
144     );
145 }
146 5 AUE_OPEN_RWTC STD {
147     int open(

```

```

148         _In_z_ const char *path,
149         int flags,
150         mode_t mode
151     );
152 }
153 ; XXX should be { int open(const char *path, int flags, ...); }
154 ; but we're not ready for varargs.
155 6 AUE_CLOSE STD|CAPENABLED {
156     int close(
157         int fd
158     );
159 }
160 7 AUE_WAIT4 STD {
161     int wait4(
162         int pid,
163         _Out_opt_ int *status,
164         int options,
165         _Out_opt_ _Contains_long_timet_ struct rusage *rusage
166     );
167 }
168 8 AUE_CREAT COMPAT {
169     int creat(
170         _In_z_ const char *path,
171         int mode
172     );
173 }
174 9 AUE_LINK STD {
175     int link(
176         _In_z_ const char *path,
177         _In_z_ const char *link
178     );
179 }
180 10 AUE_UNLINK STD {
181     int unlink(
182         _In_z_ const char *path
183     );
184 }
185 11 AUE_NULL OBSOL execv
186 12 AUE_CHDIR STD {
187     int chdir(
188         _In_z_ const char *path
189     );
190 }
191 13 AUE_FCHDIR STD {
192     int fchdir(
193         int fd
194     );
195 }
196 14 AUE_MKNOD COMPAT11 {
197     int mknod(
198         _In_z_ const char *path,
199         int mode,
200         uint32_t dev
201     );
202 }
203 15 AUE_CHMOD STD {
204     int chmod(
205         _In_z_ const char *path,
206         mode_t mode
207     );
208 }
209 16 AUE_CHOWN STD {
210     int chown(
211         _In_z_ const char *path,
212         int uid,
213         int gid
214     );
215 }
216 17 AUE_NULL STD|CAPENABLED {
217     void *break(
218         _In_ char *nsize
219     );
220 }
221 18 AUE_GETFSSTAT COMPAT4 {
222     int getfsstat(
223         _Out_writes_bytes_opt_(bufsize) _Contains_long_ struct ostatfs *buf,
224         long bufsize,
225         int mode
226     );
227 }
228 19 AUE_LSEEK COMPAT|CAPENABLED {

```

```

229         long lseek(
230             int fd,
231             long offset,
232             int whence
233         );
234     }
235 20 AUE_GETPID      STD|CAPENABLED {
236         pid_t getpid(void);
237     }
238 21 AUE_MOUNT      STD {
239         int mount(
240             _In_z_ const char *type,
241             _In_z_ const char *path,
242             int flags,
243             _In_opt_ void *data
244         );
245     }
246 22 AUE_UMOUNT     STD {
247         int unmount(
248             _In_z_ const char *path,
249             int flags
250         );
251     }
252 23 AUE_SETUID     STD|CAPENABLED {
253         int setuid(
254             uid_t uid
255         );
256     }
257 24 AUE_GETUID     STD|CAPENABLED {
258         uid_t getuid(void);
259     }
260 25 AUE_GETEUID    STD|CAPENABLED {
261         uid_t geteuid(void);
262     }
263 26 AUE_PTRACE     STD {
264         int ptrace(
265             int req,
266             pid_t pid,
267             _Inout_opt_ _Contains_long_ptr_ caddr_t addr,
268             int data
269         );
270     }
271 27 AUE_RECVMSG    STD|CAPENABLED {
272         ssize_t recvmsg(
273             int s,
274             _Inout_ _Contains_ptr_ struct msghdr *msg,
275             int flags
276         );
277     }
278 28 AUE_SENDMSG    STD|CAPENABLED {
279         ssize_t sendmsg(
280             int s,
281             _In_ _Contains_ptr_ const struct msghdr *msg,
282             int flags
283         );
284     }
285 29 AUE_RECVFROM   STD|CAPENABLED {
286         ssize_t recvfrom(
287             int s,
288             _Out_writes_bytes_(len) void *buf,
289             size_t len,
290             int flags,
291             _Out_writes_bytes_opt_(*fromlenaddr) struct sockaddr *from,
292             _Inout_opt_ __socklen_t *fromlenaddr
293         );
294     }
295 30 AUE_ACCEPT     STD|CAPENABLED {
296         int accept(
297             int s,
298             _Out_writes_bytes_opt_(*anamelen) struct sockaddr *name,
299             _Inout_opt_ __socklen_t *anamelen
300         );
301     }
302 31 AUE_GETPEERNAME STD|CAPENABLED {
303         int getpeername(
304             int fdes,
305             _Out_writes_bytes_(*alen) struct sockaddr *asa,
306             _Inout_opt_ __socklen_t *alen
307         );
308     }
309 32 AUE_GETSOCKNAME STD|CAPENABLED {

```

```

310         int getsockname(
311             int fdes,
312             _Out_writes_bytes_(*alen) struct sockaddr *asa,
313             _Inout_ __socklen_t *alen
314         );
315     }
316 33     AUE_ACCESS      STD {
317         int access(
318             _In_z_ const char *path,
319             int amode
320         );
321     }
322 34     AUE_CHFLAGS     STD {
323         int chflags(
324             _In_z_ const char *path,
325             u_long flags
326         );
327     }
328 35     AUE_FCHFLAGS    STD|CAPENABLED {
329         int fchflags(
330             int fd,
331             u_long flags
332         );
333     }
334 36     AUE_SYNC        STD|CAPENABLED {
335         int sync(void);
336     }
337 37     AUE_KILL         STD|CAPENABLED {
338         int kill(
339             int pid,
340             int signum
341         );
342     }
343 38     AUE_STAT         COMPAT {
344         int stat(
345             _In_z_ const char *path,
346             _Out_ __Contains_timet_ struct ostat *ub
347         );
348     }
349 39     AUE_GETPPID      STD|CAPENABLED {
350         pid_t getppid(void);
351     }
352 40     AUE_LSTAT        COMPAT {
353         int lstat(
354             _In_z_ const char *path,
355             _Out_ __Contains_timet_ struct ostat *ub
356         );
357     }
358 41     AUE_DUP          STD|CAPENABLED {
359         int dup(
360             u_int fd
361         );
362     }
363 42     AUE_PIPE         COMPAT10|CAPENABLED {
364         int pipe(void);
365     }
366 43     AUE_GETEGID      STD|CAPENABLED {
367         gid_t getegid(void);
368     }
369 44     AUE_PROFILE      STD|CAPENABLED {
370         int profil(
371             _Out_writes_bytes_(size) char *samples,
372             size_t size,
373             size_t offset,
374             u_int scale
375         );
376     }
377 45     AUE_KTRACE        STD {
378         int ktrace(
379             _In_z_ const char *fname,
380             int ops,
381             int facs,
382             int pid
383         );
384     }
385 46     AUE_SIGACTION    COMPAT|CAPENABLED {
386         int sigaction(
387             int signum,
388             _In_opt_ __Contains_ptr_ struct osigaction *nsa,
389             _Out_opt_ __Contains_ptr_ struct osigaction *osa
390         );

```

```

391     }
392 47     AUE_GETGID      STD|CAPENABLED {
393         gid_t getgid(void);
394     }
395 48     AUE_SIGPROCMASK COMPAT|CAPENABLED {
396         int sigprocmask(
397             int how,
398             osigset_t mask
399         );
400     }
401 ; XXX note nonstandard (bogus) calling convention - the libc stub passes
402 ; us the mask, not a pointer to it, and we return the old mask as the
403 ; (int) return value.
404 49     AUE_GETLOGIN    STD|CAPENABLED {
405         int getlogin(
406             _Out_writes_z_(namelen) char *namebuf,
407             u_int namelen
408         );
409     }
410 50     AUE_SETLOGIN    STD {
411         int setlogin(
412             _In_z_ const char *namebuf
413         );
414     }
415 51     AUE_ACCT        STD {
416         int acct(
417             _In_z_ const char *path
418         );
419     }
420 52     AUE_SIGPENDING  COMPAT|CAPENABLED {
421         int sigpending(void);
422     }
423 53     AUE_SIGALTSTACK STD|CAPENABLED {
424         int sigaltstack(
425             _In_opt_ _Contains_long_ptr_ const struct sigaltstack *ss,
426             _Out_opt_ _Contains_long_ptr_ struct sigaltstack *oss
427         );
428     }
429 54     AUE_IOCTL       STD|CAPENABLED {
430         int ioctl(
431             int fd,
432             u_long com,
433             _Inout_opt_ _Contains_long_ptr_ char *data
434         );
435     }
436 55     AUE_REBOOT      STD {
437         int reboot(
438             int opt
439         );
440     }
441 56     AUE_REVOKE      STD {
442         int revoke(
443             _In_z_ const char *path
444         );
445     }
446 57     AUE_SYMLINK     STD {
447         int symlink(
448             _In_z_ const char *path,
449             _In_z_ const char *link
450         );
451     }
452 58     AUE_READLINK   STD {
453         ssize_t readlink(
454             _In_z_ const char *path,
455             _Out_writes_z_(count) char *buf,
456             size_t count
457         );
458     }
459 59     AUE_EXECVE      STD {
460         int execve(
461             _In_z_ const char *fname,
462             _In_z_ char **argv,
463             _In_z_ char **envv
464         );
465     }
466 60     AUE_UMASK        STD|CAPENABLED {
467         mode_t umask(
468             mode_t newmask
469         );
470     }
471 61     AUE_CHROOT      STD {

```

```

472         int chroot(
473             _In_z_ const char *path
474         );
475     }
476 62     AUE_FSTAT        COMPAT|CAPENABLED {
477         int fstat(
478             int fd,
479             _Out_ _Contains_timet_ struct ostat *sb
480         );
481     }
482 63     AUE_NULL        COMPAT {
483         int getkerninfo(
484             int op,
485             _Out_writes_bytes_opt(*size) char *where,
486             _Inout_opt_ size_t *size,
487             int arg
488         );
489     }
490 64     AUE_NULL        COMPAT|CAPENABLED {
491         int getpagesize(void);
492     }
493 65     AUE_MSYNC        STD|CAPENABLED {
494         int msync(
495             _In_ void *addr,
496             size_t len,
497             int flags
498         );
499     }
500 66     AUE_VFORK        STD {
501         int vfork(void);
502     }
503 67     AUE_NULL        OBSOL    vread
504 68     AUE_NULL        OBSOL    vwrite
505 69     AUE_NULL        OBSOL    sbrk
506 70     AUE_NULL        OBSOL    sstk
507 71     AUE_MMAP        COMPAT|CAPENABLED {
508         void *mmap(
509             _In_ void *addr,
510             int len,
511             int prot,
512             int flags,
513             int fd,
514             long pos
515         );
516     }
517 72     AUE_O_VADVISE    COMPAT11 {
518         intadvise(
519             int anom
520         );
521     }
522 73     AUE_MUNMAP        STD|CAPENABLED {
523         int munmap(
524             _In_ void *addr,
525             size_t len
526         );
527     }
528 74     AUE_MPROTECT     STD|CAPENABLED {
529         int mprotect(
530             _In_ void *addr,
531             size_t len,
532             int prot
533         );
534     }
535 75     AUE_MADVISE        STD|CAPENABLED {
536         int madvise(
537             _In_ void *addr,
538             size_t len,
539             int behav
540         );
541     }
542 76     AUE_NULL        OBSOL    vhangup
543 77     AUE_NULL        OBSOL    vlimit
544 78     AUE_MINCORE        STD|CAPENABLED {
545         int mincore(
546             _In_ const void *addr,
547             size_t len,
548             _Out_writes_bytes_(len/PAGE_SIZE) char *vec
549         );
550     }
551 79     AUE_GETGROUPS     STD|CAPENABLED {
552         int getgroups(

```

```

553         int gidsetsize,
554         _Out_writes_opt_(gidsetsize) gid_t *gidset
555     );
556 }
557 80 AUE_SETGROUPS    STD {
558     int setgroups(
559         int gidsetsize,
560         _In_reads_(gidsetsize) const gid_t *gidset
561     );
562 }
563 81 AUE_GETPGRP     STD|CAPENABLED {
564     int getpgrp(void);
565 }
566 82 AUE_SETPGRP     STD {
567     int setpgid(
568         int pid,
569         int pgid
570     );
571 }
572 83 AUE_SETITIMER   STD|CAPENABLED {
573     int setitimer(
574         int which,
575         _In_ _Contains_timet_ const struct itimerval *itv,
576         _Out_opt_ _Contains_timet_ struct itimerval *oitv
577     );
578 }
579 84 AUE_WAIT4       COMPAT {
580     int wait(void);
581 }
582 85 AUE_SWAPON      STD {
583     int swapon(
584         _In_z_ const char *name
585     );
586 }
587 86 AUE_GETITIMER   STD|CAPENABLED {
588     int getitimer(
589         int which,
590         _Out_ _Contains_timet_ struct itimerval *itv
591     );
592 }
593 87 AUE_SYSCTL      COMPAT|CAPENABLED {
594     int gethostname(
595         _Out_writes_z_(len) char *hostname,
596         u_int len
597     );
598 }
599 88 AUE_SYSCTL      COMPAT {
600     int sethostname(
601         _In_reads_z_(len) char *hostname,
602         u_int len
603     );
604 }
605 89 AUE_GETDTABLESIZE STD|CAPENABLED {
606     int getdtablesize(void);
607 }
608 90 AUE_DUP2        STD|CAPENABLED {
609     int dup2(
610         u_int from,
611         u_int to
612     );
613 }
614 91 AUE_NULL        RESERVED
615 92 AUE_FCNTL        STD|CAPENABLED {
616     int fcntl(
617         int fd,
618         int cmd,
619         long arg
620     );
621 }
622 ; XXX should be { int fcntl(int fd, int cmd, ...); }
623 ; but we're not ready for varargs.
624 93 AUE_SELECT      STD|CAPENABLED {
625     int select(
626         int nd,
627         _Inout_opt_ fd_set *in,
628         _Inout_opt_ fd_set *ou,
629         _Inout_opt_ fd_set *ex,
630         _In_opt_ _Contains_long_timet_ struct timeval *tv
631     );
632 }
633 94 AUE_NULL        RESERVED

```

```

634 95     AUE_FSYNC      STD|CAPENABLED {
635         int fsync(
636             int fd
637         );
638     }
639 96     AUE_SETPRIORITY STD|CAPENABLED {
640         int setpriority(
641             int which,
642             int who,
643             int prio
644         );
645     }
646 97     AUE_SOCKET      STD|CAPENABLED {
647         int socket(
648             int domain,
649             int type,
650             int protocol
651         );
652     }
653 98     AUE_CONNECT      STD {
654         int connect(
655             int s,
656             _In_reads_bytes_(namelen) const struct sockaddr *name,
657             __socklen_t namelen
658         );
659     }
660 99     AUE_ACCEPT       COMPAT|CAPENABLED {
661         int accept(
662             int s,
663             _Out_writes_bytes_opt_(*anamelen) struct sockaddr *name,
664             __socklen_t *anamelen
665         );
666     }
667 100    AUE_GETPRIORITY  STD|CAPENABLED {
668         int getpriority(
669             int which,
670             int who
671         );
672     }
673 101    AUE_SEND         COMPAT|CAPENABLED {
674         int send(
675             int s,
676             _In_reads_bytes_(len) const void *buf,
677             int len,
678             int flags
679         );
680     }
681 102    AUE_RECV         COMPAT|CAPENABLED {
682         int recv(
683             int s,
684             _Out_writes_bytes_(len) void *buf,
685             int len,
686             int flags
687         );
688     }
689 103    AUE_SIGRETURN    COMPAT|CAPENABLED {
690         int sigreturn(
691             _In_ struct osigcontext *sigcntxp
692         );
693     }
694 104    AUE_BIND         STD {
695         int bind(
696             int s,
697             _In_reads_bytes_(namelen) const struct sockaddr *name,
698             __socklen_t namelen
699         );
700     }
701 105    AUE_SETSOCKOPT   STD|CAPENABLED {
702         int setsockopt(
703             int s,
704             int level,
705             int name,
706             _In_reads_bytes_opt_(valsize) const void *val,
707             __socklen_t valsize
708         );
709     }
710 106    AUE_LISTEN       STD|CAPENABLED {
711         int listen(
712             int s,
713             int backlog
714         );

```

```

715     }
716 107     AUE_NULL          OBSOL    vtimes
717 108     AUE_NULL          COMPAT|CAPENABLED {
718         int sigvec(
719             int signum,
720             _In_opt_ _Contains_ptr_ struct sigvec *nsv,
721             _Out_opt_ _Contains_ptr_ struct sigvec *osv
722         );
723     }
724 109     AUE_NULL          COMPAT|CAPENABLED {
725         int sigblock(
726             int mask
727         );
728     }
729 110     AUE_NULL          COMPAT|CAPENABLED {
730         int sigsetmask(
731             int mask
732         );
733     }
734 111     AUE_NULL          COMPAT|CAPENABLED {
735         int sigsuspend(
736             osigset_t mask
737         );
738     }
739 ; XXX note nonstandard (bogus) calling convention - the libc stub passes
740 ; us the mask, not a pointer to it.
741 112     AUE_NULL          COMPAT|CAPENABLED {
742         int sigstack(
743             _In_opt_ _Contains_ptr_ struct sigstack *nss,
744             _Out_opt_ _Contains_ptr_ struct sigstack *oss
745         );
746     }
747 113     AUE_RECVMSG       COMPAT|CAPENABLED {
748         int recvmsg(
749             int s,
750             _Inout_ _Contains_ptr_ struct omsghdr *msg,
751             int flags
752         );
753     }
754 114     AUE_SENDMSG       COMPAT|CAPENABLED {
755         int sendmsg(
756             int s,
757             _In_ _Contains_ptr_ const struct omsghdr *msg,
758             int flags
759         );
760     }
761 115     AUE_NULL          OBSOL    vtrace
762 116     AUE_GETTIMEOFDAY   STD|CAPENABLED {
763         int gettimeofday(
764             _Out_ _Contains_long_timet_ struct timeval *tp,
765             _Out_opt_ struct timezone *tzp
766         );
767     }
768 117     AUE_GETRUSAGE     STD|CAPENABLED {
769         int getrusage(
770             int who,
771             _Out_ _Contains_long_ struct rusage *rusage
772         );
773     }
774 118     AUE_GETSOCKOPT    STD|CAPENABLED {
775         int getsockopt(
776             int s,
777             int level,
778             int name,
779             _Out_writes_bytes_opt_(*avalsize) void *val,
780             _Inout_ __socklen_t *avalsize
781         );
782     }
783 119     AUE_NULL          RESERVED
784 120     AUE_READV          STD|CAPENABLED {
785         int readv(
786             int fd,
787             _Inout_updates_(iovcnt) _Contains_long_ptr_ struct iovec *iovp,
788             u_int iovcnt
789         );
790     }
791 121     AUE_WRITEV        STD|CAPENABLED {
792         int writev(
793             int fd,
794             _In_reads_opt_(iovcnt) _Contains_long_ptr_ struct iovec *iovp,
795             u_int iovcnt

```

```

796         );
797     }
798 122     AUE_SETTIMEOFDAY        STD {
799         int settimeofday(
800             _In_ _Contains_long_timet_ const struct timeval *tv,
801             _In_opt_ const struct timezone *tzp
802         );
803     }
804 123     AUE_FCHOWN             STD|CAPENABLED {
805         int fchown(
806             int fd,
807             int uid,
808             int gid
809         );
810     }
811 124     AUE_FCHMOD             STD|CAPENABLED {
812         int fchmod(
813             int fd,
814             mode_t mode
815         );
816     }
817 125     AUE_RECVFROM          COMPAT|CAPENABLED {
818         int recvfrom(
819             int s,
820             _Out_writes_(len) void *buf,
821             size_t len,
822             int flags,
823             _Out_writes_bytes_(*fromlenaddr) struct sockaddr *from,
824             _Inout_ __socklen_t *fromlenaddr
825         );
826     }
827 126     AUE_SETREUID          STD|CAPENABLED {
828         int setreuid(
829             int ruid,
830             int euid
831         );
832     }
833 127     AUE_SETREGID          STD|CAPENABLED {
834         int setregid(
835             int rgid,
836             int egid
837         );
838     }
839 128     AUE_RENAME            STD {
840         int rename(
841             _In_z_ const char *from,
842             _In_z_ const char *to
843         );
844     }
845 129     AUE_TRUNCATE          COMPAT {
846         int truncate(
847             _In_z_ const char *path,
848             long length
849         );
850     }
851 130     AUE_FTRUNCATE         COMPAT|CAPENABLED {
852         int ftruncate(
853             int fd,
854             long length
855         );
856     }
857 131     AUE_FLOCK             STD|CAPENABLED {
858         int flock(
859             int fd,
860             int how
861         );
862     }
863 132     AUE_MKFIFO            STD {
864         int mkfifo(
865             _In_z_ const char *path,
866             mode_t mode
867         );
868     }
869 133     AUE_SENDTO            STD|CAPENABLED {
870         ssize_t sendto(
871             int s,
872             _In_reads_bytes_(len) const void *buf,
873             size_t len,
874             int flags,
875             _In_reads_bytes_opt_(tolen) const struct sockaddr *to,
876             __socklen_t tolen

```

```

877         );
878     }
879 134 AUE_SHUTDOWN    STD|CAPENABLED {
880         int shutdown(
881             int s,
882             int how
883         );
884     }
885 135 AUE_SOCKETPAIR  STD|CAPENABLED {
886         int socketpair(
887             int domain,
888             int type,
889             int protocol,
890             _Out_writes_(2) int *rsv
891         );
892     }
893 136 AUE_MKDIR       STD {
894         int mkdir(
895             _In_z_ const char *path,
896             mode_t mode
897         );
898     }
899 137 AUE_RMDIR       STD {
900         int rmdir(
901             _In_z_ const char *path
902         );
903     }
904 138 AUE_UTIMES      STD {
905         int utimes(
906             _In_z_ const char *path,
907             _In_ _Contains_long_timet_ const struct timeval *tptr
908         );
909     }
910 139 AUE_NULL        OBSOL    sigreturn
911 140 AUE_ADJTIME      STD {
912         int adjtime(
913             _In_ _Contains_long_timet_ const struct timeval *delta,
914             _Out_opt_ _Contains_long_timet_ struct timeval *olddelta
915         );
916     }
917 141 AUE_GETPEERNAME COMPAT|CAPENABLED {
918         int getpeername(
919             int fdes,
920             _Out_writes_bytes_(*alen) struct sockaddr *asa,
921             _Inout_opt_ __socklen_t *alen
922         );
923     }
924 142 AUE_SYSCTL      COMPAT|CAPENABLED {
925         long gethostid(void);
926     }
927 143 AUE_SYSCTL      COMPAT {
928         int sethostid(
929             long hostid
930         );
931     }
932 144 AUE_GETRLIMIT   COMPAT|CAPENABLED {
933         int getrlimit(
934             u_int which,
935             _Out_ struct orlimit *rlp
936         );
937     }
938 145 AUE_SETRLIMIT   COMPAT|CAPENABLED {
939         int setrlimit(
940             u_int which,
941             _Out_ struct orlimit *rlp
942         );
943     }
944 146 AUE_KILLPG      COMPAT {
945         int killpg(
946             int pgid,
947             int signum
948         );
949     }
950 147 AUE_SETSID      STD|CAPENABLED {
951         int setsid(void);
952     }
953 148 AUE_QUOTACTL    STD {
954         int quotactl(
955             _In_z_ const char *path,
956             int cmd,
957             int uid,

```

```

958         _In_ void *arg
959     );
960 }
961 149 AUE_O_QUOTA COMPAT {
962     int quota(void);
963 }
964 150 AUE_GETSOCKNAME COMPAT|CAPENABLED {
965     int getsockname(
966         int fdes,
967         _Out_writes_bytes_(*alen) struct sockaddr *asa,
968         _Inout_ __socklen_t *alen
969     );
970 }
971 151-153 AUE_NULL RESERVED
972 ; 154 is initialised by the NLM code, if present.
973 154 AUE_NULL NOSTD {
974     int nlm_syscall(
975         int debug_level,
976         int grace_period,
977         int addr_count,
978         _In_reads_(addr_count) char **addrs
979     );
980 }
981 ; 155 is initialized by the NFS code, if present.
982 155 AUE_NFS_SVC NOSTD {
983     int nfssvc(
984         int flag,
985         _In_ void *argp
986     );
987 }
988 156 AUE_GETDIRENTRIES COMPAT|CAPENABLED {
989     int getdirentries(
990         int fd,
991         _Out_writes_bytes_(count) char *buf,
992         u_int count,
993         _Out_opt_ long *basep
994     );
995 }
996 157 AUE_STATFS COMPAT4 {
997     int statfs(
998         _In_z_ const char *path,
999         _Out_ __Contains_long_ struct ostatfs *buf
1000     );
1001 }
1002 158 AUE_FSTATFS COMPAT4|CAPENABLED {
1003     int fstatfs(
1004         int fd,
1005         _Out_ __Contains_long_ struct ostatfs *buf
1006     );
1007 }
1008 159 AUE_NULL RESERVED
1009 160 AUE_LGETFH STD {
1010     int lgetfh(
1011         _In_z_ const char *fname,
1012         _Out_ struct fhandle *fhp
1013     );
1014 }
1015 161 AUE_NFS_GETFH STD {
1016     int getfh(
1017         _In_z_ const char *fname,
1018         _Out_ struct fhandle *fhp
1019     );
1020 }
1021 162 AUE_SYSCTL COMPAT4|CAPENABLED {
1022     int getdomainname(
1023         _Out_writes_z_(len) char *domainname,
1024         int len
1025     );
1026 }
1027 163 AUE_SYSCTL COMPAT4 {
1028     int setdomainname(
1029         _In_reads_z_(len) char *domainname,
1030         int len
1031     );
1032 }
1033 164 AUE_NULL COMPAT4 {
1034     int uname(
1035         _Out_ struct utsname *name
1036     );
1037 }
1038 165 AUE_SYSARCH STD|CAPENABLED {

```

```

1039         int sysarch(
1040             int op,
1041             _In_z_ char *parms
1042         );
1043     }
1044 166     AUE_RTPRIO        STD|CAPENABLED {
1045         int rtprio(
1046             int function,
1047             pid_t pid,
1048             _Inout_ struct rtprio *rtp
1049         );
1050     }
1051 167-168 AUE_NULL        RESERVED
1052 169     AUE_SEMSYS        NOSTD {
1053         int semsys(
1054             int which,
1055             int a2,
1056             int a3,
1057             int a4,
1058             int a5
1059         );
1060     }
1061 ; XXX should be { int semsys(int which, ...); }
1062 170     AUE_MSGSYS        NOSTD {
1063         int msgsys(
1064             int which,
1065             int a2,
1066             int a3,
1067             int a4,
1068             int a5,
1069             int a6
1070         );
1071     }
1072 ; XXX should be { int msgsys(int which, ...); }
1073 171     AUE_SHMSYS        NOSTD {
1074         int shmsys(
1075             int which,
1076             int a2,
1077             int a3,
1078             int a4
1079         );
1080     }
1081 ; XXX should be { int shmsys(int which, ...); }
1082 172     AUE_NULL        RESERVED
1083 173     AUE_PREAD         COMPAT6|CAPENABLED {
1084         ssize_t pread(
1085             int fd,
1086             _Out_writes_bytes_(nbyte) void *buf,
1087             size_t nbyte,
1088             int pad,
1089             off_t offset
1090         );
1091     }
1092 174     AUE_PWRITE        COMPAT6|CAPENABLED {
1093         ssize_t pwrite(
1094             int fd,
1095             _In_reads_bytes_(nbyte) const void *buf,
1096             size_t nbyte,
1097             int pad,
1098             off_t offset
1099         );
1100     }
1101 175     AUE_SETFIB        STD {
1102         int setfib(
1103             int fibnum
1104         );
1105     }
1106 176     AUE_NTP_ADJTIME    STD {
1107         int ntp_adjtime(
1108             _Inout_ _Contains_long_ struct timex *tp
1109         );
1110     }
1111 177-180 AUE_NULL        RESERVED
1112 181     AUE_SETGID         STD|CAPENABLED {
1113         int setgid(
1114             gid_t gid
1115         );
1116     }
1117 182     AUE_SETEGID        STD|CAPENABLED {
1118         int setegid(
1119             gid_t egid

```

```

1120     );
1121 }
1122 183 AUE_SETEUID      STD|CAPENABLED {
1123     int seteuid(
1124         uid_t euid
1125     );
1126 }
1127 184 AUE_NULL        OBSOL    lfs_bmapv
1128 185 AUE_NULL        OBSOL    lfs_markv
1129 186 AUE_NULL        OBSOL    lfs_segclean
1130 187 AUE_NULL        OBSOL    lfs_segwait
1131 188 AUE_STAT        COMPAT11 {
1132     int stat(
1133         _In_z_ const char *path,
1134         _Out_ _Contains_timet_ struct freebsd11_stat *ub
1135     );
1136 }
1137 189 AUE_FSTAT        COMPAT11|CAPENABLED {
1138     int fstat(
1139         int fd,
1140         _Out_ _Contains_timet_ struct freebsd11_stat *sb
1141     );
1142 }
1143 190 AUE_LSTAT        COMPAT11 {
1144     int lstat(
1145         _In_z_ const char *path,
1146         _Out_ _Contains_timet_ struct freebsd11_stat *ub
1147     );
1148 }
1149 191 AUE_PATHCONF     STD {
1150     int pathconf(
1151         _In_z_ const char *path,
1152         int name
1153     );
1154 }
1155 192 AUE_FPATHCONF    STD|CAPENABLED {
1156     int fpathconf(
1157         int fd,
1158         int name
1159     );
1160 }
1161 193 AUE_NULL        RESERVED
1162 194 AUE_GETRLIMIT     STD|CAPENABLED {
1163     int getrlimit(
1164         u_int which,
1165         _Out_ struct rlimit *rlp
1166     );
1167 }
1168 195 AUE_SETRLIMIT     STD|CAPENABLED {
1169     int setrlimit(
1170         u_int which,
1171         _In_ struct rlimit *rlp
1172     );
1173 }
1174 196 AUE_GETDIRENTRIES COMPAT11|CAPENABLED {
1175     int getdirentries(
1176         int fd,
1177         _Out_writes_bytes_(count) char *buf,
1178         u_int count,
1179         _Out_opt_ long *basep
1180     );
1181 }
1182 197 AUE_MMAP          COMPAT6|CAPENABLED {
1183     void *mmap(
1184         _In_ void *addr,
1185         size_t len,
1186         int prot,
1187         int flags,
1188         int fd,
1189         int pad,
1190         off_t pos
1191     );
1192 }
1193 198 AUE_NULL        SYSMUX {
1194     int __syscall(
1195         int64_t number,
1196         ...
1197     );
1198 }
1199 199 AUE_LSEEK        COMPAT6|CAPENABLED {
1200     off_t lseek(

```

```

1201         int fd,
1202         int pad,
1203         off_t offset,
1204         int whence
1205     );
1206 }
1207 200 AUE_TRUNCATE COMPAT6 {
1208     int truncate(
1209         _In_z_ const char *path,
1210         int pad,
1211         off_t length
1212     );
1213 }
1214 201 AUE_FTRUNCATE COMPAT6|CAPENABLED {
1215     int ftruncate(
1216         int fd,
1217         int pad,
1218         off_t length
1219     );
1220 }
1221 202 AUE_SYSCCTL STD|CAPENABLED {
1222     int __sysctl(
1223         _In_reads_(namelen) int *name,
1224         u_int namelen,
1225         _Out_writes_bytes_opt_(*oldlenp) void *old,
1226         _Inout_opt_size_t *oldlenp,
1227         _In_reads_bytes_opt_(newlen) const void *new,
1228         size_t newlen
1229     );
1230 }
1231 203 AUE_MLOCK STD|CAPENABLED {
1232     int mlock(
1233         _In_ const void *addr,
1234         size_t len
1235     );
1236 }
1237 204 AUE_MUNLOCK STD|CAPENABLED {
1238     int munlock(
1239         _In_ const void *addr,
1240         size_t len
1241     );
1242 }
1243 205 AUE_UNDELETE STD {
1244     int undelete(
1245         _In_z_ const char *path
1246     );
1247 }
1248 206 AUE_FUTIMES STD|CAPENABLED {
1249     int futimes(
1250         int fd,
1251         _In_reads_(2) _Contains_long_timet_ const struct timeval *tptr
1252     );
1253 }
1254 207 AUE_GETPGID STD|CAPENABLED {
1255     int getpgid(
1256         pid_t pid
1257     );
1258 }
1259 208 AUE_NULL RESERVED
1260 209 AUE_POLL STD|CAPENABLED {
1261     int poll(
1262         _Inout_updates_(nfds) struct pollfd *fds,
1263         u_int nfds,
1264         int timeout
1265     );
1266 }
1267 ;
1268 ; The following are reserved for loadable syscalls
1269 ;
1270 210 AUE_NULL NODEF|NOTSTATIC lkmnosys lkmnosys nosys_args int
1271 211 AUE_NULL NODEF|NOTSTATIC lkmnosys lkmnosys nosys_args int
1272 212 AUE_NULL NODEF|NOTSTATIC lkmnosys lkmnosys nosys_args int
1273 213 AUE_NULL NODEF|NOTSTATIC lkmnosys lkmnosys nosys_args int
1274 214 AUE_NULL NODEF|NOTSTATIC lkmnosys lkmnosys nosys_args int
1275 215 AUE_NULL NODEF|NOTSTATIC lkmnosys lkmnosys nosys_args int
1276 216 AUE_NULL NODEF|NOTSTATIC lkmnosys lkmnosys nosys_args int
1277 217 AUE_NULL NODEF|NOTSTATIC lkmnosys lkmnosys nosys_args int
1278 218 AUE_NULL NODEF|NOTSTATIC lkmnosys lkmnosys nosys_args int
1279 219 AUE_NULL NODEF|NOTSTATIC lkmnosys lkmnosys nosys_args int
1280
1281 220 AUE_SEMCTL COMPAT7|NOSTD {

```

```

1282         int __semctl(
1283             int semid,
1284             int semnum,
1285             int cmd,
1286             _Contains_ptr_ union semun_old *arg
1287         );
1288     }
1289 221 AUE_SEMGET      NOSTD {
1290         int semget(
1291             key_t key,
1292             int nsems,
1293             int semflg
1294         );
1295     }
1296 222 AUE_SEMOP      NOSTD {
1297         int semop(
1298             int semid,
1299             _In_reads_(nsops) struct sembuf *sops,
1300             size_t nsops
1301         );
1302     }
1303 223 AUE_NULL        OBSOL semconfig
1304 224 AUE_MSGCTL      COMPAT7|NOSTD {
1305         int msgctl(
1306             int msqid,
1307             int cmd,
1308             _Contains_long_ptr_timet_ struct msqid_ds_old *buf
1309         );
1310     }
1311 225 AUE_MSGGET      NOSTD {
1312         int msgget(
1313             key_t key,
1314             int msgflg
1315         );
1316     }
1317 226 AUE_MSGSND      NOSTD {
1318         int msgsnd(
1319             int msqid,
1320             _In_reads_bytes_(msgsz) _Contains_long_ const void *msgp,
1321             size_t msgsz,
1322             int msgflg
1323         );
1324     }
1325 227 AUE_MSGRCV      NOSTD {
1326         ssize_t msgrcv(
1327             int msqid,
1328             _Out_writes_bytes_(msgsz) _Contains_long_ void *msgp,
1329             size_t msgsz,
1330             long msgtyp,
1331             int msgflg
1332         );
1333     }
1334 228 AUE_SHMAT       NOSTD {
1335         void *shmat(
1336             int shmid,
1337             _In_ const void *shmaddr,
1338             int shmflg
1339         );
1340     }
1341 229 AUE_SHMCTL      COMPAT7|NOSTD {
1342         int shmctl(
1343             int shmid,
1344             int cmd,
1345             _Inout_opt_ _Contains_long_ struct shmids_old *buf
1346         );
1347     }
1348 230 AUE_SHMDT       NOSTD {
1349         int shmdt(
1350             _In_ const void *shmaddr
1351         );
1352     }
1353 231 AUE_SHMGET      NOSTD {
1354         int shmget(
1355             key_t key,
1356             size_t size,
1357             int shmflg
1358         );
1359     }
1360 232 AUE_NULL        STD|CAPENABLED {
1361         int clock_gettime(
1362             clockid_t clock_id,

```

```

1363         _Out_ _Contains_long_timet_ struct timespec *tp
1364     );
1365 }
1366 233 AUE_CLOCK_SETTIME      STD {
1367     int clock_settime(
1368         clockid_t clock_id,
1369         _In_ _Contains_long_timet_ const struct timespec *tp
1370     );
1371 }
1372 234 AUE_NULL              STD|CAPENABLED {
1373     int clock_getres(
1374         clockid_t clock_id,
1375         _Out_ _Contains_long_timet_ struct timespec *tp
1376     );
1377 }
1378 235 AUE_NULL              STD|CAPENABLED {
1379     int ktimer_create(
1380         clockid_t clock_id,
1381         _In_ _Contains_long_ptr_ struct sigevent *evp,
1382         _Out_ int *timerid
1383     );
1384 }
1385 236 AUE_NULL              STD|CAPENABLED {
1386     int ktimer_delete(
1387         int timerid
1388     );
1389 }
1390 237 AUE_NULL              STD|CAPENABLED {
1391     int ktimer_settime(
1392         int timerid,
1393         int flags,
1394         _In_ _Contains_long_timet_ const struct itimerspec *value,
1395         _Out_opt_ _Contains_long_timet_ struct itimerspec *ovalue
1396     );
1397 }
1398 238 AUE_NULL              STD|CAPENABLED {
1399     int ktimer_gettime(
1400         int timerid,
1401         _Out_ _Contains_long_timet_ struct itimerspec *value
1402     );
1403 }
1404 239 AUE_NULL              STD|CAPENABLED {
1405     int ktimer_getoverrun(
1406         int timerid
1407     );
1408 }
1409 240 AUE_NULL              STD|CAPENABLED {
1410     int nanosleep(
1411         _In_ _Contains_long_timet_ const struct timespec *rqtp,
1412         _Out_opt_ _Contains_long_timet_ struct timespec *rmtp
1413     );
1414 }
1415 241 AUE_NULL              STD {
1416     int ffclock_getcounter(
1417         _Out_ ffclock_t *ffcount
1418     );
1419 }
1420 242 AUE_NULL              STD {
1421     int ffclock_setestimate(
1422         _In_ _Contains_timet_ struct ffclock_estimate *cest
1423     );
1424 }
1425 243 AUE_NULL              STD {
1426     int ffclock_getestimate(
1427         _Out_ _Contains_timet_ struct ffclock_estimate *cest
1428     );
1429 }
1430 244 AUE_NULL              STD {
1431     int clock_nanosleep(
1432         clockid_t clock_id,
1433         int flags,
1434         _In_ _Contains_long_timet_ const struct timespec *rqtp,
1435         _Out_opt_ _Contains_long_timet_ struct timespec *rmtp
1436     );
1437 }
1438 245-246 AUE_NULL          RESERVED
1439 247 AUE_NULL              STD {
1440     int clock_getcpuclockid2(
1441         id_t id,
1442         int which,
1443         _Out_ clockid_t *clock_id

```

```

1444         );
1445     }
1446 248     AUE_NULL          STD|CAPENABLED {
1447         int ntp_gettime(
1448             _Out_ _Contains_long_timet_ struct ntp_timeval *ntvp
1449         );
1450     }
1451 249     AUE_NULL          RESERVED
1452 250     AUE_MINHERIT      STD|CAPENABLED {
1453         int minherit(
1454             _In_ void *addr,
1455             size_t len,
1456             int inherit
1457         );
1458     }
1459 251     AUE_RFORK         STD {
1460         int rfork(
1461             int flags
1462         );
1463     }
1464 252     AUE_POLL          OBSOL    openbsd_poll
1465 253     AUE_ISSETUGID     STD|CAPENABLED {
1466         int issetugid(void);
1467     }
1468 254     AUE_LCHOWN        STD {
1469         int lchown(
1470             _In_z_ const char *path,
1471             int uid,
1472             int gid
1473         );
1474     }
1475 255     AUE_AIO_READ      STD|CAPENABLED {
1476         int aio_read(
1477             _Inout_ _Contains_long_ptr_ struct aiocb *aiocbp
1478         );
1479     }
1480 256     AUE_AIO_WRITE     STD|CAPENABLED {
1481         int aio_write(
1482             _Inout_ _Contains_long_ptr_ struct aiocb *aiocbp
1483         );
1484     }
1485 257     AUE_LIO_LISTIO   STD|CAPENABLED {
1486         int lio_listio(
1487             int mode,
1488             _Inout_updates_(nent) _Contains_long_ptr_ struct aiocb * const *acb_list,
1489             int nent,
1490             _In_opt_ _Contains_long_ptr_ struct sigevent *sig
1491         );
1492     }
1493 258-271 AUE_NULL        RESERVED
1494 272     AUE_O_GETDENTS    COMPAT11|CAPENABLED {
1495         int getdents(
1496             int fd,
1497             _Out_writes_bytes_(count) char *buf,
1498             size_t count
1499         );
1500     }
1501 273     AUE_NULL          RESERVED
1502 274     AUE_LCHMOD         STD {
1503         int lchmod(
1504             _In_z_ const char *path,
1505             mode_t mode
1506         );
1507     }
1508 275     AUE_NULL          OBSOL    netbsd_lchown
1509 276     AUE_LUTIMES        STD {
1510         int lutimes(
1511             _In_z_ const char *path,
1512             _In_ _Contains_long_timet_ const struct timeval *tptr
1513         );
1514     }
1515 277     AUE_NULL          OBSOL    netbsd_msync
1516 278     AUE_STAT           COMPAT11 {
1517         int nstat(
1518             _In_z_ const char *path,
1519             _Out_ _Contains_long_timet_ struct nstat *ub
1520         );
1521     }
1522 279     AUE_FSTAT         COMPAT11 {
1523         int nfstat(
1524             int fd,

```

```

1525         _Out_ _Contains_long_timet_ struct nstat *sb
1526     );
1527 }
1528 280 AUE_LSTAT COMPAT11 {
1529     int nlstat(
1530         _In_z_ const char *path,
1531         _Out_ _Contains_long_timet_ struct nstat *ub
1532     );
1533 }
1534 281-288 AUE_NULL RESERVED
1535 289 AUE_PREADV STD|CAPENABLED {
1536     ssize_t preadv(
1537         int fd,
1538         _In_reads_(iovcnt) _Contains_long_ptr_ struct iovec *iovp,
1539         u_int iovcnt,
1540         off_t offset
1541     );
1542 }
1543 290 AUE_PWRITEV STD|CAPENABLED {
1544     ssize_t pwritev(
1545         int fd,
1546         _In_reads_(iovcnt) _Contains_long_ptr_ struct iovec *iovp,
1547         u_int iovcnt,
1548         off_t offset
1549     );
1550 }
1551 291-296 AUE_NULL RESERVED
1552 297 AUE_FHSTATFS COMPAT4 {
1553     int fhstatfs(
1554         _In_ const struct fhandle *u_fhp,
1555         _Out_ _Contains_long_ struct ostatfs *buf
1556     );
1557 }
1558 298 AUE_FHOPEN STD {
1559     int fhopen(
1560         _In_ const struct fhandle *u_fhp,
1561         int flags
1562     );
1563 }
1564 299 AUE_FHSTAT COMPAT11 {
1565     int fhstat(
1566         _In_ const struct fhandle *u_fhp,
1567         _Out_ _Contains_long_timet_ struct freebsd11_stat *sb
1568     );
1569 }
1570 300 AUE_NULL STD {
1571     int modnext(
1572         int modid
1573     );
1574 }
1575 301 AUE_NULL STD {
1576     int modstat(
1577         int modid,
1578         _Out_ _Contains_long_ struct module_stat *stat
1579     );
1580 }
1581 302 AUE_NULL STD {
1582     int modfnnext(
1583         int modid
1584     );
1585 }
1586 303 AUE_NULL STD {
1587     int modfind(
1588         _In_z_ const char *name
1589     );
1590 }
1591 304 AUE_MODLOAD STD {
1592     int kldload(
1593         _In_z_ const char *file
1594     );
1595 }
1596 305 AUE_MODUNLOAD STD {
1597     int kldunload(
1598         int fileid
1599     );
1600 }
1601 306 AUE_NULL STD {
1602     int kldfind(
1603         _In_z_ const char *file
1604     );
1605 }

```

```

1606 307  AUE_NULL      STD {
1607         int kldnext(
1608             int fileid
1609         );
1610     }
1611 308  AUE_NULL      STD {
1612         int kldstat(
1613             int fileid,
1614             _Out_ _Contains_long_ptr_ struct kld_file_stat *stat
1615         );
1616     }
1617 309  AUE_NULL      STD {
1618         int kldfirstmod(
1619             int fileid
1620         );
1621     }
1622 310  AUE_GETSID    STD|CAPENABLED {
1623         int getsid(
1624             pid_t pid
1625         );
1626     }
1627 311  AUE_SETRESUID  STD|CAPENABLED {
1628         int setresuid(
1629             uid_t ruid,
1630             uid_t euid,
1631             uid_t suid
1632         );
1633     }
1634 312  AUE_SETRESGID  STD|CAPENABLED {
1635         int setresgid(
1636             gid_t rgid,
1637             gid_t egid,
1638             gid_t sgid
1639         );
1640     }
1641 313  AUE_NULL      OBSOL  signanosleep
1642 314  AUE_AIO_RETURN STD|CAPENABLED {
1643         ssize_t aio_return(
1644             _Inout_ _Contains_long_ptr_ struct aiocb *aiocbp
1645         );
1646     }
1647 315  AUE_AIO_SUSPEND STD|CAPENABLED {
1648         int aio_suspend(
1649             _Inout_updates_(nent) _Contains_long_ptr_ struct aiocb * const * aiocbp,
1650             int nent,
1651             _In_opt_ _Contains_long_timet_ const struct timespec *timeout
1652         );
1653     }
1654 316  AUE_AIO_CANCEL STD|CAPENABLED {
1655         int aio_cancel(
1656             int fd,
1657             _In_opt_ _Contains_long_ptr_ struct aiocb *aiocbp
1658         );
1659     }
1660 317  AUE_AIO_ERROR  STD|CAPENABLED {
1661         int aio_error(
1662             _In_ _Contains_long_ptr_ struct aiocb *aiocbp
1663         );
1664     }
1665 318  AUE_AIO_READ    COMPAT6|CAPENABLED {
1666         int aio_read(
1667             _Inout_ _Contains_long_ptr_ struct oaiocb *aiocbp
1668         );
1669     }
1670 319  AUE_AIO_WRITE    COMPAT6|CAPENABLED {
1671         int aio_write(
1672             _Inout_ _Contains_long_ptr_ struct oaiocb *aiocbp
1673         );
1674     }
1675 320  AUE_LIO_LISTIO  COMPAT6|CAPENABLED {
1676         int lio_listio(
1677             int mode,
1678             _Inout_updates_(nent) _Contains_long_ptr_ struct oaiocb * const *acb_list,
1679             int nent,
1680             _In_opt_ _Contains_ptr_ struct osigevent *sig
1681         );
1682     }
1683 321  AUE_NULL      STD|CAPENABLED {
1684         int yield(void);
1685     }
1686 322  AUE_NULL      OBSOL  thr_sleep

```

```

1687 323 AUE_NULL      OBSOL thr_wakeup
1688 324 AUE_MLOCKALL  STD|CAPENABLED {
1689      int mlockall(
1690          int how
1691      );
1692 }
1693 325 AUE_MUNLOCKALL STD|CAPENABLED {
1694      int munlockall(void);
1695 }
1696 326 AUE_GETCWD   STD {
1697      int __getcwd(
1698          _Out_writes_z_(buflen) char *buf,
1699          size_t buflen
1700      );
1701 }
1702 327 AUE_NULL      STD|CAPENABLED {
1703      int sched_setparam(
1704          pid_t pid,
1705          _In_ const struct sched_param *param
1706      );
1707 }
1708 328 AUE_NULL      STD|CAPENABLED {
1709      int sched_getparam(
1710          pid_t pid,
1711          _Out_ struct sched_param *param
1712      );
1713 }
1714 329 AUE_NULL      STD|CAPENABLED {
1715      int sched_setscheduler(
1716          pid_t pid,
1717          int policy,
1718          _In_ const struct sched_param *param
1719      );
1720 }
1721 330 AUE_NULL      STD|CAPENABLED {
1722      int sched_getscheduler(
1723          pid_t pid
1724      );
1725 }
1726 331 AUE_NULL      STD|CAPENABLED {
1727      int sched_yield(void);
1728 }
1729 332 AUE_NULL      STD|CAPENABLED {
1730      int sched_get_priority_max(
1731          int policy
1732      );
1733 }
1734 333 AUE_NULL      STD|CAPENABLED {
1735      int sched_get_priority_min(
1736          int policy
1737      );
1738 }
1739 334 AUE_NULL      STD|CAPENABLED {
1740      int sched_rr_get_interval(
1741          pid_t pid,
1742          _Out_ _Contains_long_timet_ struct timespec *interval
1743      );
1744 }
1745 335 AUE_NULL      STD|CAPENABLED {
1746      int utrace(
1747          _In_reads_bytes_(len) const void *addr,
1748          size_t len
1749      );
1750 }
1751 336 AUE_SENDFILE  COMPAT4|CAPENABLED {
1752      int sendfile(
1753          int fd,
1754          int s,
1755          off_t offset,
1756          size_t nbytes,
1757          _In_opt_ _Contains_ptr_ struct sf_hdr *hdr,
1758          _Out_opt_ off_t *sbytes,
1759          int flags
1760      );
1761 }
1762 337 AUE_NULL      STD {
1763      int kldsym(
1764          int fileid,
1765          int cmd,
1766          _In_ _Contains_long_ptr_ void *data
1767      );

```

```

1768     }
1769 338 AUE_JAIL      STD {
1770     int jail(
1771         _In_ _Contains_ptr_ struct jail *jail
1772     );
1773     }
1774 339 AUE_NULL      NOSTD|NOTSTATIC {
1775     int nnpfs_syscall(
1776         int operation,
1777         char *a_pathP,
1778         int a_opcode,
1779         void *a_paramsP,
1780         int a_followSymlinks
1781     );
1782     }
1783 340 AUE_SIGPROCMASK STD|CAPENABLED {
1784     int sigprocmask(
1785         int how,
1786         _In_opt_ const sigset_t *set,
1787         _Out_opt_ sigset_t *oset
1788     );
1789     }
1790 341 AUE_SIGSUSPEND STD|CAPENABLED {
1791     int sigsuspend(
1792         _In_ const sigset_t *sigmask
1793     );
1794     }
1795 342 AUE_SIGACTION  COMPAT4|CAPENABLED {
1796     int sigaction(
1797         int sig,
1798         _In_opt_ _Contains_ptr_ const struct sigaction *act,
1799         _Out_opt_ _Contains_ptr_ struct sigaction *oact
1800     );
1801     }
1802 343 AUE_SIGPENDING STD|CAPENABLED {
1803     int sigpending(
1804         _In_ sigset_t *set
1805     );
1806     }
1807 344 AUE_SIGRETURN  COMPAT4|CAPENABLED {
1808     int sigreturn(
1809         _In_ _Contains_long_ptr_ const struct freebsd4_ucontext *sigcntxp
1810     );
1811     }
1812 345 AUE_SIGWAIT    STD|CAPENABLED {
1813     int sigtimedwait(
1814         _In_ const sigset_t *set,
1815         _Out_opt_ _Contains_long_ptr_ struct siginfo *info,
1816         _In_opt_ _Contains_long_timet_ const struct timespec *timeout
1817     );
1818     }
1819 346 AUE_NULL      STD|CAPENABLED {
1820     int sigwaitinfo(
1821         _In_ const sigset_t *set,
1822         _Out_opt_ _Contains_long_ptr_ struct siginfo *info
1823     );
1824     }
1825 347 AUE_ACL_GET_FILE STD {
1826     int __acl_get_file(
1827         _In_z_ const char *path,
1828         acl_type_t type,
1829         _Out_ struct acl *aclp
1830     );
1831     }
1832 348 AUE_ACL_SET_FILE STD {
1833     int __acl_set_file(
1834         _In_z_ const char *path,
1835         acl_type_t type,
1836         _In_ struct acl *aclp
1837     );
1838     }
1839 349 AUE_ACL_GET_FD STD|CAPENABLED {
1840     int __acl_get_fd(
1841         int filedес,
1842         acl_type_t type,
1843         _Out_ struct acl *aclp
1844     );
1845     }
1846 350 AUE_ACL_SET_FD STD|CAPENABLED {
1847     int __acl_set_fd(
1848         int filedес,

```

```

1849         acl_type_t type,
1850         _In_ struct acl *aclp
1851     );
1852 }
1853 351 AUE_ACL_DELETE_FILE    STD {
1854     int __acl_delete_file(
1855         _In_z_ const char *path,
1856         acl_type_t type
1857     );
1858 }
1859 352 AUE_ACL_DELETE_FD     STD|CAPENABLED {
1860     int __acl_delete_fd(
1861         int filedes,
1862         acl_type_t type
1863     );
1864 }
1865 353 AUE_ACL_CHECK_FILE    STD {
1866     int __acl_aclcheck_file(
1867         _In_z_ const char *path,
1868         acl_type_t type,
1869         _In_ struct acl *aclp
1870     );
1871 }
1872 354 AUE_ACL_CHECK_FD     STD|CAPENABLED {
1873     int __acl_aclcheck_fd(
1874         int filedes,
1875         acl_type_t type,
1876         _In_ struct acl *aclp
1877     );
1878 }
1879 355 AUE_EXTATTRCTL    STD {
1880     int extattrctl(
1881         _In_z_ const char *path,
1882         int cmd,
1883         _In_z_opt_ const char *filename,
1884         int attrnamespace,
1885         _In_z_ const char *attrname
1886     );
1887 }
1888 356 AUE_EXTATTR_SET_FILE    STD {
1889     ssize_t extattr_set_file(
1890         _In_z_ const char *path,
1891         int attrnamespace,
1892         _In_z_ const char *attrname,
1893         _In_reads_bytes_(nbytes) void *data,
1894         size_t nbytes
1895     );
1896 }
1897 357 AUE_EXTATTR_GET_FILE    STD {
1898     ssize_t extattr_get_file(
1899         _In_z_ const char *path,
1900         int attrnamespace,
1901         _In_z_ const char *attrname,
1902         _Out_writes_bytes_(nbytes) void *data,
1903         size_t nbytes
1904     );
1905 }
1906 358 AUE_EXTATTR_DELETE_FILE STD {
1907     int extattr_delete_file(
1908         _In_z_ const char *path,
1909         int attrnamespace,
1910         _In_z_ const char *attrname
1911     );
1912 }
1913 359 AUE_AIO_WAITCOMPLETE STD|CAPENABLED {
1914     ssize_t aio_waitcomplete(
1915         _Outptr_result_maybenull_ struct aiocb **aiocbp,
1916         _In_opt_ _Contains_long_timet_ struct timespec *timeout
1917     );
1918 }
1919 360 AUE_GETRESUID    STD|CAPENABLED {
1920     int getresuid(
1921         _Out_opt_ uid_t *ruid,
1922         _Out_opt_ uid_t *euid,
1923         _Out_opt_ uid_t *suid
1924     );
1925 }
1926 361 AUE_GETRESGID    STD|CAPENABLED {
1927     int getresgid(
1928         _Out_opt_ gid_t *rgid,
1929         _Out_opt_ gid_t *egid,

```

```

1930         _Out_opt_ gid_t *sgid
1931     );
1932 }
1933 362 AUE_KQUEUE      STD|CAPENABLED {
1934         int kqueue(void);
1935 }
1936 363 AUE_KEVENT      COMPAT11|CAPENABLED {
1937         int kevent(
1938             int fd,
1939             _In_reads_opt_(nchanges) _Contains_ptr_ const struct freebsd11_kevent *changelist,
1940             int nchanges,
1941             _Out_writes_opt_(nevents) _Contains_ptr_ struct freebsd11_kevent *eventlist,
1942             int nevents,
1943             _In_opt_ _Contains_long_timet_ const struct timespec *timeout
1944         );
1945 }
1946 364 AUE_NULL        OBSOL    __cap_get_proc
1947 365 AUE_NULL        OBSOL    __cap_set_proc
1948 366 AUE_NULL        OBSOL    __cap_get_fd
1949 367 AUE_NULL        OBSOL    __cap_get_file
1950 368 AUE_NULL        OBSOL    __cap_set_fd
1951 369 AUE_NULL        OBSOL    __cap_set_file
1952 370 AUE_NULL        RESERVED
1953 371 AUE_EXTATTR_SET_FD  STD|CAPENABLED {
1954         ssize_t extattr_set_fd(
1955             int fd,
1956             int attrnamespace,
1957             _In_z_ const char *attrname,
1958             _In_reads_bytes_(nbytes) void *data,
1959             size_t nbytes
1960         );
1961 }
1962 372 AUE_EXTATTR_GET_FD  STD|CAPENABLED {
1963         ssize_t extattr_get_fd(
1964             int fd,
1965             int attrnamespace,
1966             _In_z_ const char *attrname,
1967             _Out_writes_bytes_(nbytes) void *data,
1968             size_t nbytes
1969         );
1970 }
1971 373 AUE_EXTATTR_DELETE_FD STD|CAPENABLED {
1972         int extattr_delete_fd(
1973             int fd,
1974             int attrnamespace,
1975             _In_z_ const char *attrname
1976         );
1977 }
1978 374 AUE_SETUGID      STD {
1979         int __setugid(
1980             int flag
1981         );
1982 }
1983 375 AUE_NULL        OBSOL    nfsc1nt
1984 376 AUE_EACCESS      STD {
1985         int eaccess(
1986             _In_z_ const char *path,
1987             int amode
1988         );
1989 }
1990 377 AUE_NULL        NOSTD|NOTSTATIC {
1991         int afs3_syscall(
1992             long syscall,
1993             long parm1,
1994             long parm2,
1995             long parm3,
1996             long parm4,
1997             long parm5,
1998             long parm6
1999         );
2000 }
2001 378 AUE_NMOUNT       STD {
2002         int nmount(
2003             _In_reads_(iovcnt) _Contains_long_ptr_ struct iovec *iovp,
2004             unsigned int iovcnt,
2005             int flags
2006         );
2007 }
2008 379 AUE_NULL        OBSOL    kse_exit
2009 380 AUE_NULL        OBSOL    kse_wakeup
2010 381 AUE_NULL        OBSOL    kse_create

```

```

2011 382 AUE_NULL      OBSOL   kse_thr_interrupt
2012 383 AUE_NULL      OBSOL   kse_release
2013 384 AUE_NULL      STD|CAPENABLED {
2014      int __mac_get_proc(
2015          _In_ _Contains_long_ptr_ struct mac *mac_p
2016      );
2017 }
2018 385 AUE_NULL      STD|CAPENABLED {
2019      int __mac_set_proc(
2020          _In_ _Contains_long_ptr_ struct mac *mac_p
2021      );
2022 }
2023 386 AUE_NULL      STD|CAPENABLED {
2024      int __mac_get_fd(
2025          int fd,
2026          _In_ _Contains_long_ptr_ struct mac *mac_p
2027      );
2028 }
2029 387 AUE_NULL      STD {
2030      int __mac_get_file(
2031          _In_z_ const char *path_p,
2032          _In_ _Contains_long_ptr_ struct mac *mac_p
2033      );
2034 }
2035 388 AUE_NULL      STD|CAPENABLED {
2036      int __mac_set_fd(
2037          int fd,
2038          _In_ _Contains_long_ptr_ struct mac *mac_p
2039      );
2040 }
2041 389 AUE_NULL      STD {
2042      int __mac_set_file(
2043          _In_z_ const char *path_p,
2044          _In_ _Contains_long_ptr_ struct mac *mac_p
2045      );
2046 }
2047 390 AUE_NULL      STD {
2048      int kenv(
2049          int what,
2050          _In_z_opt_ const char *name,
2051          _Inout_updates_opt_(len) char *value,
2052          int len
2053      );
2054 }
2055 391 AUE_LCHFLAGS  STD {
2056      int lchflags(
2057          _In_z_ const char *path,
2058          u_long flags
2059      );
2060 }
2061 392 AUE_NULL      STD|CAPENABLED {
2062      int uuidgen(
2063          _Out_writes_(count) struct uuid *store,
2064          int count
2065      );
2066 }
2067 393 AUE_SENDFILE  STD|CAPENABLED {
2068      int sendfile(
2069          int fd,
2070          int s,
2071          off_t offset,
2072          size_t nbytes,
2073          _In_opt_ _Contains_ptr_ struct sf_hdr *hdr,
2074          _Out_opt_ off_t *sbytes,
2075          int flags
2076      );
2077 }
2078 394 AUE_NULL      STD {
2079      int mac_syscall(
2080          _In_z_ const char *policy,
2081          int call,
2082          _In_opt_ void *arg
2083      );
2084 }
2085 395 AUE_GETFSSTAT COMPAT11 {
2086      int getfsstat(
2087          _Out_writes_bytes_opt_(bufsize) struct freebsd11_statfs *buf,
2088          long bufsize,
2089          int mode
2090      );
2091 }

```

```

2092 396  AUE_STATFS      COMPAT11 {
2093         int statfs(
2094             _In_z_ const char *path,
2095             _Out_ struct freebsd11_statfs *buf
2096         );
2097     }
2098 397  AUE_FSTATFS     COMPAT11|CAPENABLED {
2099         int fstatfs(
2100             int fd,
2101             _Out_ struct freebsd11_statfs *buf
2102         );
2103     }
2104 398  AUE_FHSTATFS    COMPAT11 {
2105         int fhstatfs(
2106             _In_ const struct fhandle *u_fhp,
2107             _Out_ struct freebsd11_statfs *buf
2108         );
2109     }
2110 399  AUE_NULL        RESERVED
2111 400  AUE_SEM_CLOSE    NOSTD {
2112         int ksem_close(
2113             semid_t id
2114         );
2115     }
2116 401  AUE_SEM_POST     NOSTD {
2117         int ksem_post(
2118             semid_t id
2119         );
2120     }
2121 402  AUE_SEM_WAIT     NOSTD {
2122         int ksem_wait(
2123             semid_t id
2124         );
2125     }
2126 403  AUE_SEM_TRYWAIT  NOSTD {
2127         int ksem_trywait(
2128             semid_t id
2129         );
2130     }
2131 404  AUE_SEM_INIT     NOSTD {
2132         int ksem_init(
2133             _Out_ semid_t *idp,
2134             unsigned int value
2135         );
2136     }
2137 405  AUE_SEM_OPEN     NOSTD {
2138         int ksem_open(
2139             _Out_ semid_t *idp,
2140             _In_z_ const char *name,
2141             int oflag,
2142             mode_t mode,
2143             unsigned int value
2144         );
2145     }
2146 406  AUE_SEM_UNLINK   NOSTD {
2147         int ksem_unlink(
2148             _In_z_ const char *name
2149         );
2150     }
2151 407  AUE_SEM_GETVALUE NOSTD {
2152         int ksem_getvalue(
2153             semid_t id,
2154             _Out_ int *val
2155         );
2156     }
2157 408  AUE_SEM_DESTROY  NOSTD {
2158         int ksem_destroy(
2159             semid_t id
2160         );
2161     }
2162 409  AUE_NULL        STD {
2163         int __mac_get_pid(
2164             pid_t pid,
2165             _In_ _Contains_long_ptr_ struct mac *mac_p
2166         );
2167     }
2168 410  AUE_NULL        STD {
2169         int __mac_get_link(
2170             _In_z_ const char *path_p,
2171             _In_ _Contains_long_ptr_ struct mac *mac_p
2172         );

```

```

2173     }
2174 411     AUE_NULL          STD {
2175         int __mac_set_link(
2176             _In_z_ const char *path_p,
2177             _In_ _Contains_long_ptr_ struct mac *mac_p
2178         );
2179     }
2180 412     AUE_EXTATTR_SET_LINK    STD {
2181         ssize_t extattr_set_link(
2182             _In_z_ const char *path,
2183             int attrnamespace,
2184             _In_z_ const char *attrname,
2185             _In_reads_bytes_(nbytes) void *data,
2186             size_t nbytes
2187         );
2188     }
2189 413     AUE_EXTATTR_GET_LINK    STD {
2190         ssize_t extattr_get_link(
2191             _In_z_ const char *path,
2192             int attrnamespace,
2193             _In_z_ const char *attrname,
2194             _Out_writes_bytes_(nbytes) void *data,
2195             size_t nbytes
2196         );
2197     }
2198 414     AUE_EXTATTR_DELETE_LINK STD {
2199         int extattr_delete_link(
2200             _In_z_ const char *path,
2201             int attrnamespace,
2202             _In_z_ const char *attrname
2203         );
2204     }
2205 415     AUE_NULL          STD {
2206         int __mac_execve(
2207             _In_z_ const char *fname,
2208             _In_ char **argv,
2209             _In_ char **envv,
2210             _In_ _Contains_long_ptr_ struct mac *mac_p
2211         );
2212     }
2213 416     AUE_SIGACTION    STD|CAPENABLED {
2214         int sigaction(
2215             int sig,
2216             _In_opt_ _Contains_ptr_ const struct sigaction *act,
2217             _Out_opt_ _Contains_ptr_ struct sigaction *oact
2218         );
2219     }
2220 417     AUE_SIGRETURN    STD|CAPENABLED {
2221         int sigreturn(
2222             _In_ _Contains_long_ptr_ const struct __ucontext *sigcntxp
2223         );
2224     }
2225 418-420 AUE_NULL          RESERVED
2226 421     AUE_NULL          STD|CAPENABLED {
2227         int getcontext(
2228             _Out_ _Contains_long_ptr_ struct __ucontext *ucp
2229         );
2230     }
2231 422     AUE_NULL          STD|CAPENABLED {
2232         int setcontext(
2233             _In_ _Contains_long_ptr_ const struct __ucontext *ucp
2234         );
2235     }
2236 423     AUE_NULL          STD {
2237         int swapcontext(
2238             _Out_ _Contains_long_ptr_ struct __ucontext *oucp,
2239             _In_ _Contains_long_ptr_ const struct __ucontext *ucp
2240         );
2241     }
2242 424     AUE_SWAPOFF        COMPAT13 {
2243         int swapoff(
2244             _In_z_ const char *name
2245         );
2246     }
2247 425     AUE_ACL_GET_LINK    STD {
2248         int __acl_get_link(
2249             _In_z_ const char *path,
2250             acl_type_t type,
2251             _Out_ struct acl *aclp
2252         );
2253     }

```

```

2254 426  AUE_ACL_SET_LINK      STD {
2255         int __acl_set_link(
2256             _In_z_ const char *path,
2257             acl_type_t type,
2258             _In_ struct acl *aclp
2259         );
2260     }
2261 427  AUE_ACL_DELETE_LINK   STD {
2262         int __acl_delete_link(
2263             _In_z_ const char *path,
2264             acl_type_t type
2265         );
2266     }
2267 428  AUE_ACL_CHECK_LINK    STD {
2268         int __acl_aclcheck_link(
2269             _In_z_ const char *path,
2270             acl_type_t type,
2271             _In_ struct acl *aclp
2272         );
2273     }
2274 429  AUE_SIGWAIT          STD|CAPENABLED {
2275         int sigwait(
2276             _In_ const sigset_t *set,
2277             _Out_ int *sig
2278         );
2279     }
2280 430  AUE_THR_CREATE        STD|CAPENABLED {
2281         int thr_create(
2282             _In_ _Contains_long_ptr_ ucontext_t *ctx,
2283             _Out_ long *id,
2284             int flags
2285         );
2286     }
2287 431  AUE_THR_EXIT          STD|CAPENABLED {
2288         void thr_exit(
2289             _Out_opt_ long *state
2290         );
2291     }
2292 432  AUE_NULL              STD|CAPENABLED {
2293         int thr_self(
2294             _Out_ long *id
2295         );
2296     }
2297 433  AUE_THR_KILL          STD|CAPENABLED {
2298         int thr_kill(
2299             long id,
2300             int sig
2301         );
2302     }
2303
2304 434  AUE_NULL              COMPAT10 {
2305         int _umtx_lock(
2306             _Inout_ struct umtx *umtx
2307         );
2308     }
2309
2310 435  AUE_NULL              COMPAT10 {
2311         int _umtx_unlock(
2312             _Inout_ struct umtx *umtx
2313         );
2314     }
2315
2316 436  AUE_JAIL_ATTACH        STD {
2317         int jail_attach(
2318             int jid
2319         );
2320     }
2321 437  AUE_EXTATTR_LIST_FD    STD|CAPENABLED {
2322         ssize_t extattr_list_fd(
2323             int fd,
2324             int attrnamespace,
2325             _Out_writes_bytes_opt_(nbytes) void *data,
2326             size_t nbytes
2327         );
2328     }
2329 438  AUE_EXTATTR_LIST_FILE  STD {
2330         ssize_t extattr_list_file(
2331             _In_z_ const char *path,
2332             int attrnamespace,
2333             _Out_writes_bytes_opt_(nbytes) void *data,
2334             size_t nbytes

```

```

2335         );
2336     }
2337 439     AUE_EXTATTR_LIST_LINK    STD {
2338         ssize_t extattr_list_link(
2339             _In_z_ const char *path,
2340             int attrnamespace,
2341             _Out_writes_bytes_opt_(nbytes) void *data,
2342             size_t nbytes
2343         );
2344     }
2345 440     AUE_NULL                OBSOL    kse_switchin
2346 441     AUE_SEMWAIT             NOSTD {
2347         int ksem_timedwait(
2348             semid_t id,
2349             _In_opt_ _Contains_long_timet_ const struct timespec *abstime
2350         );
2351     }
2352 442     AUE_NULL                STD|CAPENABLED {
2353         int thr_suspend(
2354             _In_opt_ _Contains_long_timet_ const struct timespec *timeout
2355         );
2356     }
2357 443     AUE_NULL                STD|CAPENABLED {
2358         int thr_wake(
2359             long id
2360         );
2361     }
2362 444     AUE_MODUNLOAD          STD {
2363         int kldunloadf(
2364             int fileid,
2365             int flags
2366         );
2367     }
2368 445     AUE_AUDIT              STD {
2369         int audit(
2370             _In_reads_bytes_(length) const void *record,
2371             u_int length
2372         );
2373     }
2374 446     AUE_AUDITON            STD {
2375         int auditon(
2376             int cmd,
2377             _In_opt_ void *data,
2378             u_int length
2379         );
2380     }
2381 447     AUE_GETAUID            STD|CAPENABLED {
2382         int getauid(
2383             _Out_ uid_t *auid
2384         );
2385     }
2386 448     AUE_SETAUID            STD|CAPENABLED {
2387         int setauid(
2388             _In_ uid_t *auid
2389         );
2390     }
2391 449     AUE_GETAUDIT           STD|CAPENABLED {
2392         int getaudit(
2393             _Out_ struct auditinfo *auditinfo
2394         );
2395     }
2396 450     AUE_SETAUDIT           STD|CAPENABLED {
2397         int setaudit(
2398             _In_ struct auditinfo *auditinfo
2399         );
2400     }
2401 451     AUE_GETAUDIT_ADDR       STD|CAPENABLED {
2402         int getaudit_addr(
2403             _Out_writes_bytes_(length) struct auditinfo_addr *auditinfo_addr,
2404             u_int length
2405         );
2406     }
2407 452     AUE_SETAUDIT_ADDR       STD|CAPENABLED {
2408         int setaudit_addr(
2409             _In_reads_bytes_(length) struct auditinfo_addr *auditinfo_addr,
2410             u_int length
2411         );
2412     }
2413 453     AUE_AUDITCTL            STD {
2414         int auditctl(
2415             _In_z_ const char *path

```

```

2416         );
2417     }
2418 454 AUE_NULL        STD|CAPENABLED {
2419         int _umtx_op(
2420             _Inout_ void *obj,
2421             int op,
2422             u_long val,
2423             _In_ void *uaddr1,
2424             _In_ void *uaddr2
2425         );
2426     }
2427 455 AUE_THR_NEW     STD|CAPENABLED {
2428         int thr_new(
2429             _In_ _Contains_long_ptr_ struct thr_param *param,
2430             int param_size
2431         );
2432     }
2433 456 AUE_NULL        STD|CAPENABLED {
2434         int sigqueue(
2435             pid_t pid,
2436             int signum,
2437             _In_ void *value
2438         );
2439     }
2440
2441 457 AUE_MQ_OPEN      NOSTD {
2442         int kmq_open(
2443             _In_z_ const char *path,
2444             int flags,
2445             mode_t mode,
2446             _In_opt_ _Contains_long_ const struct mq_attr *attr
2447         );
2448     }
2449 458 AUE_MQ_SETATTR   NOSTD|CAPENABLED {
2450         int kmq_setattr(
2451             int mqd,
2452             _In_opt_ _Contains_long_ const struct mq_attr *attr,
2453             _Out_opt_ _Contains_long_ struct mq_attr *oattr
2454         );
2455     }
2456 459 AUE_MQ_TIMEDRECEIVE NOSTD|CAPENABLED {
2457         int kmq_timedreceive(
2458             int mqd,
2459             _Out_writes_bytes_(msg_len) char *msg_ptr,
2460             size_t msg_len,
2461             _Out_opt_ unsigned *msg_prio,
2462             _In_opt_ _Contains_long_timet_ const struct timespec *abs_timeout
2463         );
2464     }
2465 460 AUE_MQ_TIMEDSEND NOSTD|CAPENABLED {
2466         int kmq_timedsend(
2467             int mqd,
2468             _In_reads_bytes_(msg_len) const char *msg_ptr,
2469             size_t msg_len,
2470             unsigned msg_prio,
2471             _In_opt_ _Contains_long_timet_ const struct timespec *abs_timeout
2472         );
2473     }
2474 461 AUE_MQ_NOTIFY     NOSTD|CAPENABLED {
2475         int kmq_notify(
2476             int mqd,
2477             _In_opt_ _Contains_long_ptr_ const struct sigevent *sigev
2478         );
2479     }
2480 462 AUE_MQ_UNLINK     NOSTD {
2481         int kmq_unlink(
2482             _In_z_ const char *path
2483         );
2484     }
2485 463 AUE_NULL        STD|CAPENABLED {
2486         void abort2(
2487             _In_z_ const char *why,
2488             int nargs,
2489             _In_reads_(nargs) void **args
2490         );
2491     }
2492 464 AUE_NULL        STD|CAPENABLED {
2493         int thr_set_name(
2494             long id,
2495             _In_z_ const char *name
2496         );

```

```

2497     }
2498 465     AUE_AIO_FSYNC     STD|CAPENABLED {
2499         int aio_fsync(
2500             int op,
2501             _In_ _Contains_long_ptr_ struct aiocb *aiocbp
2502         );
2503     }
2504 466     AUE_RTPRIO       STD|CAPENABLED {
2505         int rtprio_thread(
2506             int function,
2507             lwpid_t lwpid,
2508             _Inout_ struct rtprio *rtp
2509         );
2510     }
2511 467-470 AUE_NULL        RESERVED
2512 471     AUE_SCTP_PEELOFF NOSTD|CAPENABLED {
2513         int sctp_peeloff(
2514             int sd,
2515             uint32_t name
2516         );
2517     }
2518 472     AUE_SCTP_GENERIC_SENDMSG NOSTD|CAPENABLED {
2519         int sctp_generic_sendmsg(
2520             int sd,
2521             _In_reads_bytes_(mlen) void *msg,
2522             int mlen,
2523             _In_reads_bytes_(tolen) const struct sockaddr *to,
2524             __socklen_t tolen,
2525             _In_opt_ struct sctp_sndrcvinfo *sinfo,
2526             int flags
2527         );
2528     }
2529 473     AUE_SCTP_GENERIC_SENDMSG_IOV NOSTD|CAPENABLED {
2530         int sctp_generic_sendmsg_iov(
2531             int sd,
2532             _In_reads_(iovlen) _Contains_long_ptr_ struct iovec *iov,
2533             int iovlen,
2534             _In_reads_bytes_(tolen) const struct sockaddr *to,
2535             __socklen_t tolen,
2536             _In_opt_ struct sctp_sndrcvinfo *sinfo,
2537             int flags
2538         );
2539     }
2540 474     AUE_SCTP_GENERIC_RECVMSG NOSTD|CAPENABLED {
2541         int sctp_generic_recvmsg(
2542             int sd,
2543             _In_reads_(iovlen) _Contains_long_ptr_ struct iovec *iov,
2544             int iovlen,
2545             _Out_writes_bytes_(*fromlenaddr) struct sockaddr *from,
2546             _Out_ __socklen_t *fromlenaddr,
2547             _In_opt_ struct sctp_sndrcvinfo *sinfo,
2548             _Out_opt_ int *msg_flags
2549         );
2550     }
2551 475     AUE_PREAD        STD|CAPENABLED {
2552         ssize_t pread(
2553             int fd,
2554             _Out_writes_bytes_(nbyte) void *buf,
2555             size_t nbyte,
2556             off_t offset
2557         );
2558     }
2559 476     AUE_PWRITE       STD|CAPENABLED {
2560         ssize_t pwrite(
2561             int fd,
2562             _In_reads_bytes_(nbyte) const void *buf,
2563             size_t nbyte,
2564             off_t offset
2565         );
2566     }
2567 477     AUE_MMAP         STD|CAPENABLED {
2568         void *mmap(
2569             _In_ void *addr,
2570             size_t len,
2571             int prot,
2572             int flags,
2573             int fd,
2574             off_t pos
2575         );
2576     }
2577 478     AUE_LSEEK        STD|CAPENABLED {

```

```

2578         off_t lseek(
2579             int fd,
2580             off_t offset,
2581             int whence
2582         );
2583     }
2584 479     AUE_TRUNCATE    STD {
2585         int truncate(
2586             _In_z_ const char *path,
2587             off_t length
2588         );
2589     }
2590 480     AUE_FTRUNCATE  STD|CAPENABLED {
2591         int ftruncate(
2592             int fd,
2593             off_t length
2594         );
2595     }
2596 481     AUE_THR_KILL2  STD {
2597         int thr_kill2(
2598             pid_t pid,
2599             long id,
2600             int sig
2601         );
2602     }
2603 482     AUE_SHMOPEN    COMPAT12|CAPENABLED {
2604         int shm_open(
2605             _In_z_ const char *path,
2606             int flags,
2607             mode_t mode
2608         );
2609     }
2610 483     AUE_SHMUNLINK  STD {
2611         int shm_unlink(
2612             _In_z_ const char *path
2613         );
2614     }
2615 484     AUE_NULL       STD {
2616         int cpuset(
2617             _Out_ cpusetid_t *setid
2618         );
2619     }
2620 485     AUE_NULL       STD {
2621         int cpuset_setid(
2622             cpuwhich_t which,
2623             id_t id,
2624             cpusetid_t setid
2625         );
2626     }
2627 486     AUE_NULL       STD {
2628         int cpuset_getid(
2629             cpulevel_t level,
2630             cpuwhich_t which,
2631             id_t id,
2632             _Out_ cpusetid_t *setid
2633         );
2634     }
2635 487     AUE_NULL       STD|CAPENABLED {
2636         int cpuset_getaffinity(
2637             cpulevel_t level,
2638             cpuwhich_t which,
2639             id_t id,
2640             size_t cpusetsize,
2641             _Out_ cpuset_t *mask
2642         );
2643     }
2644 488     AUE_NULL       STD|CAPENABLED {
2645         int cpuset_setaffinity(
2646             cpulevel_t level,
2647             cpuwhich_t which,
2648             id_t id,
2649             size_t cpusetsize,
2650             _Out_ const cpuset_t *mask
2651         );
2652     }
2653 489     AUE_FACCESSAT  STD|CAPENABLED {
2654         int faccessat(
2655             int fd,
2656             _In_z_ const char *path,
2657             int amode,
2658             int flag

```

```

2659         );
2660     }
2661 490     AUE_FCHMODAT     STD|CAPENABLED {
2662         int fchmodat(
2663             int fd,
2664             _In_z_ const char *path,
2665             mode_t mode,
2666             int flag
2667         );
2668     }
2669 491     AUE_FCHOWNAT     STD|CAPENABLED {
2670         int fchownat(
2671             int fd,
2672             _In_z_ const char *path,
2673             uid_t uid,
2674             gid_t gid,
2675             int flag
2676         );
2677     }
2678 492     AUE_FEXECVE     STD|CAPENABLED {
2679         int fexecve(
2680             int fd,
2681             _In_ char **argv,
2682             _In_ char **envv
2683         );
2684     }
2685 493     AUE_FSTATAT     COMPAT11|CAPENABLED {
2686         int fstatat(
2687             int fd,
2688             _In_z_ const char *path,
2689             _Out_ Contains_long_timet_ struct freebsd11_stat *buf,
2690             int flag
2691         );
2692     }
2693 494     AUE_FUTIMESAT     STD|CAPENABLED {
2694         int futimesat(
2695             int fd,
2696             _In_z_ const char *path,
2697             _In_reads_(2) _Contains_long_timet_ const struct timeval *times
2698         );
2699     }
2700 495     AUE_LINKAT     STD|CAPENABLED {
2701         int linkat(
2702             int fd1,
2703             _In_z_ const char *path1,
2704             int fd2,
2705             _In_z_ const char *path2,
2706             int flag
2707         );
2708     }
2709 496     AUE_MKDIRAT     STD|CAPENABLED {
2710         int mkdirat(
2711             int fd,
2712             _In_z_ const char *path,
2713             mode_t mode
2714         );
2715     }
2716 497     AUE_MKFIFOAT     STD|CAPENABLED {
2717         int mkfifoat(
2718             int fd,
2719             _In_z_ const char *path,
2720             mode_t mode
2721         );
2722     }
2723 498     AUE_MKNODAT     COMPAT11|CAPENABLED {
2724         int mknodat(
2725             int fd,
2726             _In_z_ const char *path,
2727             mode_t mode,
2728             uint32_t dev
2729         );
2730     }
2731 ; XXX: see the comment for open
2732 499     AUE_OPENAT_RWTC STD|CAPENABLED {
2733         int openat(
2734             int fd,
2735             _In_z_ const char *path,
2736             int flag,
2737             mode_t mode
2738         );
2739     }

```

```

2740 500  AUE_READLINKAT  STD|CAPENABLED {
2741         ssize_t readlinkat(
2742             int fd,
2743             _In_z_ const char *path,
2744             _Out_writes_bytes_(bufsize) char *buf,
2745             size_t bufsize
2746         );
2747     }
2748 501  AUE_RENAMEAT    STD|CAPENABLED {
2749         int renameat(
2750             int oldfd,
2751             _In_z_ const char *old,
2752             int newfd,
2753             _In_z_ const char *new
2754         );
2755     }
2756 502  AUE_SYMLINKAT   STD|CAPENABLED {
2757         int symlinkat(
2758             _In_z_ const char *path1,
2759             int fd,
2760             _In_z_ const char *path2
2761         );
2762     }
2763 503  AUE_UNLINKAT    STD|CAPENABLED {
2764         int unlinkat(
2765             int fd,
2766             _In_z_ const char *path,
2767             int flag
2768         );
2769     }
2770 504  AUE_POSIX_OPENPT  STD {
2771         int posix_openpt(
2772             int flags
2773         );
2774     }
2775 ; 505 is initialised by the kgssapi code, if present.
2776 505  AUE_NULL        NOSTD {
2777         int gssd_syscall(
2778             _In_z_ const char *path
2779         );
2780     }
2781 506  AUE_JAIL_GET     STD {
2782         int jail_get(
2783             _In_reads_(iovcnt) _Contains_long_ptr_ struct iovec *iovp,
2784             unsigned int iovcnt,
2785             int flags
2786         );
2787     }
2788 507  AUE_JAIL_SET     STD {
2789         int jail_set(
2790             _In_reads_(iovcnt) _Contains_long_ptr_ struct iovec *iovp,
2791             unsigned int iovcnt,
2792             int flags
2793         );
2794     }
2795 508  AUE_JAIL_REMOVE  STD {
2796         int jail_remove(
2797             int jid
2798         );
2799     }
2800 509  AUE_CLOSEFROM   COMPAT12|CAPENABLED {
2801         int closefrom(
2802             int lowfd
2803         );
2804     }
2805 510  AUE_SEMCTL       NOSTD {
2806         int __semctl(
2807             int semid,
2808             int semnum,
2809             int cmd,
2810             _Inout_ _Contains_ptr_ union semun *arg
2811         );
2812     }
2813 511  AUE_MSGCTL       NOSTD {
2814         int msgctl(
2815             int msqid,
2816             int cmd,
2817             _Inout_opt_ _Contains_long_ptr_ struct msqid_ds *buf
2818         );
2819     }
2820 512  AUE_SHMCTL       NOSTD {

```

```

2821         int shmctl(
2822             int shmid,
2823             int cmd,
2824             _Inout_opt_ _Contains_long_ struct shmids *buf
2825         );
2826     }
2827 513 AUE_LPATHCONF    STD {
2828         int lpathconf(
2829             _In_z_ const char *path,
2830             int name
2831         );
2832     }
2833 514 AUE_NULL        OBSOL    cap_new
2834 515 AUE_CAP_RIGHTS_GET    STD|CAPENABLED {
2835         int __cap_rights_get(
2836             int version,
2837             int fd,
2838             _Out_ cap_rights_t *rightsp
2839         );
2840     }
2841 516 AUE_CAP_ENTER    STD|CAPENABLED {
2842         int cap_enter(void);
2843     }
2844 517 AUE_CAP_GETMODE    STD|CAPENABLED {
2845         int cap_getmode(
2846             _Out_ u_int *modep
2847         );
2848     }
2849 518 AUE_PDFORK        STD|CAPENABLED {
2850         int pdfork(
2851             _Out_ int *fdp,
2852             int flags
2853         );
2854     }
2855 519 AUE_PDKILL        STD|CAPENABLED {
2856         int pdkill(
2857             int fd,
2858             int signum
2859         );
2860     }
2861 520 AUE_PDGETPID        STD|CAPENABLED {
2862         int pdgetpid(
2863             int fd,
2864             _Out_ pid_t *pidp
2865         );
2866     }
2867 521 AUE_NULL        RESERVED
2868 522 AUE_SELECT        STD|CAPENABLED {
2869         int pselect(
2870             int nd,
2871             _Inout_opt_ fd_set *in,
2872             _Inout_opt_ fd_set *ou,
2873             _Inout_opt_ fd_set *ex,
2874             _In_opt_ _Contains_long_timet_ const struct timespec *ts,
2875             _In_opt_ const sigset_t *sm
2876         );
2877     }
2878 523 AUE_GETLOGINCLASS    STD|CAPENABLED {
2879         int getloginclass(
2880             _Out_writes_z_(namelen) char *namebuf,
2881             size_t namelen
2882         );
2883     }
2884 524 AUE_SETLOGINCLASS    STD {
2885         int setloginclass(
2886             _In_z_ const char *namebuf
2887         );
2888     }
2889 525 AUE_NULL        STD {
2890         int rctl_get_racct(
2891             _In_reads_bytes_(inbuflen) const void *inbufp,
2892             size_t inbuflen,
2893             _Out_writes_bytes_(outbuflen) void *outbufp,
2894             size_t outbuflen
2895         );
2896     }
2897 526 AUE_NULL        STD {
2898         int rctl_get_rules(
2899             _In_reads_bytes_(inbuflen) const void *inbufp,
2900             size_t inbuflen,
2901             _Out_writes_bytes_(outbuflen) void *outbufp,

```

```

2902         size_t outbuflen
2903     );
2904 }
2905 527 AUE_NULL      STD {
2906     int rctl_get_limits(
2907         _In_reads_bytes_(inbuflen) const void *inbufp,
2908         size_t inbuflen,
2909         _Out_writes_bytes_(outbuflen) void *outbufp,
2910         size_t outbuflen
2911     );
2912 }
2913 528 AUE_NULL      STD {
2914     int rctl_add_rule(
2915         _In_reads_bytes_(inbuflen) const void *inbufp,
2916         size_t inbuflen,
2917         _Out_writes_bytes_(outbuflen) void *outbufp,
2918         size_t outbuflen
2919     );
2920 }
2921 529 AUE_NULL      STD {
2922     int rctl_remove_rule(
2923         _In_reads_bytes_(inbuflen) const void *inbufp,
2924         size_t inbuflen,
2925         _Out_writes_bytes_(outbuflen) void *outbufp,
2926         size_t outbuflen
2927     );
2928 }
2929 530 AUE_POSIX_FALLOCATE STD|CAPENABLED {
2930     int posix_fallocate(
2931         int fd,
2932         off_t offset,
2933         off_t len
2934     );
2935 }
2936 531 AUE_POSIX_FADVISE  STD|CAPENABLED {
2937     int posix_fadvise(
2938         int fd,
2939         off_t offset,
2940         off_t len,
2941         int advice
2942     );
2943 }
2944 532 AUE_WAIT6      STD {
2945     int wait6(
2946         idtype_t idtype,
2947         id_t id,
2948         _Out_opt_ int *status,
2949         int options,
2950         _Out_opt_ _Contains_long_ struct __wrusage *wrusage,
2951         _Out_opt_ _Contains_long_ptr_ struct siginfo *info
2952     );
2953 }
2954 533 AUE_CAP_RIGHTS_LIMIT STD|CAPENABLED {
2955     int cap_rights_limit(
2956         int fd,
2957         _In_ cap_rights_t *rightsp
2958     );
2959 }
2960 534 AUE_CAP_IOCTL_LIMIT STD|CAPENABLED {
2961     int cap_ioctl_limit(
2962         int fd,
2963         _In_reads_(ncmds) const u_long *cmds,
2964         size_t ncmds
2965     );
2966 }
2967 535 AUE_CAP_IOCTL_GET  STD|CAPENABLED {
2968     ssize_t cap_ioctl_get(
2969         int fd,
2970         _Out_writes_(maxcmds) u_long *cmds,
2971         size_t maxcmds
2972     );
2973 }
2974 536 AUE_CAP_FCNTLS_LIMIT STD|CAPENABLED {
2975     int cap_fcntls_limit(
2976         int fd,
2977         uint32_t fcntlrights
2978     );
2979 }
2980 537 AUE_CAP_FCNTLS_GET  STD|CAPENABLED {
2981     int cap_fcntls_get(
2982         int fd,

```

```

2983         _Out_ uint32_t *fcntlrightsp
2984     );
2985 }
2986 538 AUE_BINDAT      STD|CAPENABLED {
2987     int bindat(
2988         int fd,
2989         int s,
2990         _In_reads_bytes_(namelen) const struct sockaddr *name,
2991         __socklen_t namelen
2992     );
2993 }
2994 539 AUE_CONNECTAT  STD|CAPENABLED {
2995     int connectat(
2996         int fd,
2997         int s,
2998         _In_reads_bytes_(namelen) const struct sockaddr *name,
2999         __socklen_t namelen
3000     );
3001 }
3002 540 AUE_CHFLAGSAT  STD|CAPENABLED {
3003     int chflagsat(
3004         int fd,
3005         _In_z_ const char *path,
3006         u_long flags,
3007         int atflag
3008     );
3009 }
3010 541 AUE_ACCEPT     STD|CAPENABLED {
3011     int accept4(
3012         int s,
3013         _Out_writes_bytes_opt_(*anamelen) struct sockaddr *name,
3014         _Inout_opt_ __socklen_t *anamelen,
3015         int flags
3016     );
3017 }
3018 542 AUE_PIPE       STD|CAPENABLED {
3019     int pipe2(
3020         _Out_writes_(2) int *fildes,
3021         int flags
3022     );
3023 }
3024 543 AUE_AIO_MLOCK   STD {
3025     int aio_mlock(
3026         _In_ _Contains_long_ptr_ struct aiocb *aiocbp
3027     );
3028 }
3029 544 AUE_PROCCTL     STD {
3030     int procctl(
3031         idtype_t idtype,
3032         id_t id,
3033         int com,
3034         _In_opt_ void *data
3035     );
3036 }
3037 545 AUE_POLL        STD|CAPENABLED {
3038     int ppoll(
3039         _Inout_updates_(nfds) struct pollfd *fds,
3040         u_int nfds,
3041         _In_opt_ _Contains_long_timet_ const struct timespec *ts,
3042         _In_opt_ const sigset_t *set
3043     );
3044 }
3045 546 AUE_FUTIMES     STD|CAPENABLED {
3046     int futimens(
3047         int fd,
3048         _In_reads_(2) _Contains_long_timet_ const struct timespec *times
3049     );
3050 }
3051 547 AUE_FUTIMESAT   STD|CAPENABLED {
3052     int utimensat(
3053         int fd,
3054         _In_z_ const char *path,
3055         _In_reads_(2) _Contains_long_timet_ const struct timespec *times,
3056         int flag
3057     );
3058 }
3059 548 AUE_NULL        OBSOL    numa_getaffinity
3060 549 AUE_NULL        OBSOL    numa_setaffinity
3061 550 AUE_FSYNC        STD|CAPENABLED {
3062     int fdatsync(
3063         int fd

```

```

3064         );
3065     }
3066 551     AUE_FSTAT      STD|CAPENABLED {
3067         int fstat(
3068             int fd,
3069             _Out_ _Contains_long_timet_ struct stat *sb
3070         );
3071     }
3072 552     AUE_FSTATAT   STD|CAPENABLED {
3073         int fstatat(
3074             int fd,
3075             _In_z_ const char *path,
3076             _Out_ _Contains_long_timet_ struct stat *buf,
3077             int flag
3078         );
3079     }
3080 553     AUE_FHSTAT    STD {
3081         int fhstat(
3082             _In_ const struct fhandle *u_fhp,
3083             _Out_ _Contains_long_timet_ struct stat *sb
3084         );
3085     }
3086 554     AUE_GETDIRENTRIES STD|CAPENABLED {
3087         ssize_t getdirentries(
3088             int fd,
3089             _Out_writes_bytes_(count) char *buf,
3090             size_t count,
3091             _Out_opt_ off_t *basep
3092         );
3093     }
3094 555     AUE_STATFS     STD {
3095         int statfs(
3096             _In_z_ const char *path,
3097             _Out_ struct statfs *buf
3098         );
3099     }
3100 556     AUE_FSTATFS    STD|CAPENABLED {
3101         int fstatfs(
3102             int fd,
3103             _Out_ struct statfs *buf
3104         );
3105     }
3106 557     AUE_GETFSSTAT  STD {
3107         int getfsstat(
3108             _Out_writes_bytes_opt_(bufsize) struct statfs *buf,
3109             long bufsize,
3110             int mode
3111         );
3112     }
3113 558     AUE_FHSTATFS   STD {
3114         int fhstatfs(
3115             _In_ const struct fhandle *u_fhp,
3116             _Out_ struct statfs *buf
3117         );
3118     }
3119 559     AUE_MKNODAT    STD|CAPENABLED {
3120         int mknodat(
3121             int fd,
3122             _In_z_ const char *path,
3123             mode_t mode,
3124             dev_t dev
3125         );
3126     }
3127 560     AUE_KEVENT      STD|CAPENABLED {
3128         int kevent(
3129             int fd,
3130             _In_reads_opt_(nchanges) _Contains_ptr_ const struct kevent *changelist,
3131             int nchanges,
3132             _Out_writes_opt_(nevents) _Contains_ptr_ struct kevent *eventlist,
3133             int nevents,
3134             _In_opt_ _Contains_long_timet_ const struct timespec *timeout
3135         );
3136     }
3137 561     AUE_NULL        STD|CAPENABLED {
3138         int cpuset_getdomain(
3139             cpulevel_t level,
3140             cpuwhich_t which,
3141             id_t id,
3142             size_t domainsetsize,
3143             _Out_writes_bytes_(domainsetsize) domainset_t *mask,
3144             _Out_ int *policy

```

```

3145     );
3146 }
3147 562 AUE_NULL      STD|CAPENABLED {
3148     int cpuset_setdomain(
3149         cpulevel_t level,
3150         cpuwhich_t which,
3151         id_t id,
3152         size_t domainsetsize,
3153         _In_ domainset_t *mask,
3154         int policy
3155     );
3156 }
3157 563 AUE_NULL      STD|CAPENABLED {
3158     int getrandom(
3159         _Out_writes_bytes_(buflen) void *buf,
3160         size_t buflen,
3161         unsigned int flags
3162     );
3163 }
3164 564 AUE_NULL      STD {
3165     int getfhat(
3166         int fd,
3167         _In_z_ char *path,
3168         _Out_ struct fhandle *fhp,
3169         int flags
3170     );
3171 }
3172 565 AUE_NULL      STD {
3173     int fhlink(
3174         _In_ struct fhandle *fhp,
3175         _In_z_ const char *to
3176     );
3177 }
3178 566 AUE_NULL      STD {
3179     int fhlinkat(
3180         _In_ struct fhandle *fhp,
3181         int tofd,
3182         _In_z_ const char *to,
3183     );
3184 }
3185 567 AUE_NULL      STD {
3186     int fhreadlink(
3187         _In_ struct fhandle *fhp,
3188         _Out_writes_(bufsize) char *buf,
3189         size_t bufsize
3190     );
3191 }
3192 568 AUE_UNLINKAT  STD|CAPENABLED {
3193     int funlinkat(
3194         int dfd,
3195         _In_z_ const char *path,
3196         int fd,
3197         int flag
3198     );
3199 }
3200 569 AUE_NULL      STD|CAPENABLED {
3201     ssize_t copy_file_range(
3202         int infd,
3203         _Inout_opt_ off_t *inoffp,
3204         int outfd,
3205         _Inout_opt_ off_t *outoffp,
3206         size_t len,
3207         unsigned int flags
3208     );
3209 }
3210 570 AUE_SYSCTL     STD|CAPENABLED {
3211     int __sysctlbyname(
3212         _In_reads_(namelen) const char *name,
3213         size_t namelen,
3214         _Out_writes_bytes_opt_(*oldlenp) void *old,
3215         _Inout_opt_ size_t *oldlenp,
3216         _In_reads_bytes_opt_(newlen) void *new,
3217         size_t newlen
3218     );
3219 }
3220 571 AUE_SHMOPEN    STD|CAPENABLED {
3221     int shm_open2(
3222         _In_z_ const char *path,
3223         int flags,
3224         mode_t mode,
3225         int shmflags,

```

```

3226         _In_z_ const char *name
3227     );
3228 }
3229 572 AUE_SHMRENAME STD {
3230     int shm_rename(
3231         _In_z_ const char *path_from,
3232         _In_z_ const char *path_to,
3233         int flags
3234     );
3235 }
3236 573 AUE_NULL STD|CAPENABLED {
3237     int sigfastblock(
3238         int cmd,
3239         _Inout_opt_ uint32_t *ptr
3240     );
3241 }
3242 574 AUE_REALPATHAT STD {
3243     int __realpathat(
3244         int fd,
3245         _In_z_ const char *path,
3246         _Out_writes_z_(size) char *buf,
3247         size_t size,
3248         int flags
3249     );
3250 }
3251 575 AUE_CLOSERANGE STD|CAPENABLED {
3252     int close_range(
3253         u_int lowfd,
3254         u_int highfd,
3255         int flags
3256     );
3257 }
3258 ; 576 is initialised by the krpc code, if present.
3259 576 AUE_NULL NOSTD {
3260     int rpctl_syscall(
3261         int op,
3262         _In_z_ const char *path
3263     );
3264 }
3265 577 AUE_SPECIALFD STD|CAPENABLED {
3266     int __specialfd(
3267         int type,
3268         _In_reads_bytes_(len) const void *req,
3269         size_t len
3270     );
3271 }
3272 578 AUE_AIO_WRITEV STD|CAPENABLED {
3273     int aio_writev(
3274         _Inout_ _Contains_long_ptr_ struct aiocb *aiocbp
3275     );
3276 }
3277 579 AUE_AIO_READV STD|CAPENABLED {
3278     int aio_readv(
3279         _Inout_ _Contains_long_ptr_ struct aiocb *aiocbp
3280     );
3281 }
3282 580 AUE_FSPACECTL STD|CAPENABLED {
3283     int fspacectl(
3284         int fd,
3285         int cmd,
3286         _In_ const struct spacectl_range *rqsr,
3287         int flags,
3288         _Out_opt_ struct spacectl_range *rmsr,
3289     );
3290 }
3291 581 AUE_NULL STD|CAPENABLED {
3292     int sched_getcpu(void);
3293 }
3294 582 AUE_SWAPOFF STD {
3295     int swapoff(
3296         _In_z_ const char *name,
3297         u_int flags,
3298     );
3299 }
3300 583 AUE_KQUEUE STD|CAPENABLED {
3301     int kqueuex(
3302         u_int flags
3303     );
3304 }
3305 584 AUE_NULL STD|CAPENABLED {
3306     int membarrier(

```

```
3307         int cmd,
3308         unsigned flags,
3309         int cpu_id
3310     );
3311 }
3312 585 AUE_TIMERFD      STD|CAPENABLED {
3313         int timerfd_create(
3314             int clockid,
3315             int flags
3316         );
3317 }
3318 586 AUE_TIMERFD      STD|CAPENABLED {
3319         int timerfd_gettime(
3320             int fd,
3321             _Out_ _Contains_long_timet_ struct itimerspec *curr_value
3322         );
3323 }
3324 587 AUE_TIMERFD      STD|CAPENABLED {
3325         int timerfd_settime(
3326             int fd,
3327             int flags,
3328             _In_ _Contains_long_timet_ const struct itimerspec *new_value,
3329             _Out_opt_ _Contains_long_timet_ struct itimerspec *old_value
3330         );
3331     }
3332
3333
3334 ; vim: syntax=off
```

generated by cgkit v1.2.3 (git 2.25.1) at 2023-12-11 14:46:26 +0000